

DoubleAgents: Human-Agent Alignment in a Socially Embedded Workflow

Tao Long*
long@cs.columbia.edu
Columbia University
New York, NY, USA

Xuanming Zhang*
billyzhang@cs.columbia.edu
Columbia University
New York, NY, USA

Sitong Wang
sitong@cs.columbia.edu
Columbia University
New York, NY, USA

Zhou Yu
zy2461@columbia.edu
Columbia University
New York, NY, USA

Lydia B. Chilton
chilton@cs.columbia.edu
Columbia University
New York, NY, USA

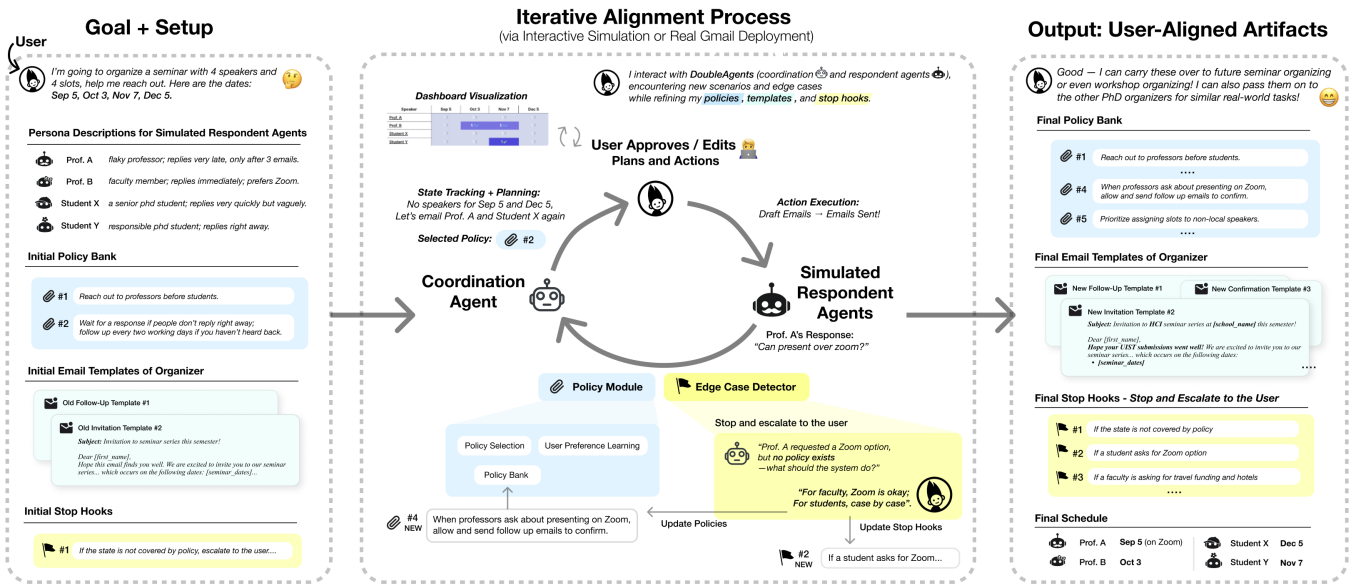


Figure 1: DoubleAgents aligns agentic AI with user preferences for socially embedded coordination tasks such as organizing seminars. From an initial setup (left), including a goal, simulated personas, and seed policies and templates, users iteratively align the system (middle) by reviewing proposed plans, approving actions, handling edge cases, and refining policies, templates, and stop hooks across repeated runs. This produces (right) both a finalized speaker-slot allocation and a set of user-aligned artifacts — policies, email templates, and stop hooks — that persist and transfer to future coordination tasks.

Abstract

Aligning agentic AI with user intent is critical for delegating complex, socially embedded tasks, yet user preferences are often implicit, evolving, and difficult to specify upfront. We present DoubleAgents, a system for human-agent alignment in coordination tasks, grounded in distributed cognition. DoubleAgents integrates three components: (1) a coordination agent that maintains state

and proposes plans and actions, (2) a dashboard visualization that makes the agent's reasoning legible for user evaluation, and (3) a policy module that transforms user edits into reusable alignment artifacts, including coordination policies, email templates, and stop hooks, which improve system behavior over time. We evaluate DoubleAgents through a two-day in-lab interactive simulation study ($n=10$), three real-world deployments, and a technical evaluation. Participants' comfort in offloading tasks and reliance on DoubleAgents both increased over time, correlating with the three distributed cognition components. Participants still required control at points of uncertainty — edge-case flagging and context-dependent actions. We contribute a distributed cognition approach to human-agent alignment in socially embedded tasks. We further introduce interactive simulation as a methodological testbed for rapid iteration and alignment testing of agentic systems.

*Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License.
HCOMP 2026, Alexandria, VA, USA

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2894-5/2026/09
<https://doi.org/10.1145/3834580.3838737>

CCS Concepts

• **Human-centered computing** → **Interactive systems and tools**.

Keywords

generative AI, AI agents, trust, reliance, transparency, state tracking, seminar organizing, control, human coordination, constitutional AI

ACM Reference Format:

Tao Long, Xuanming Zhang, Sitong Wang, Zhou Yu, and Lydia B. Chilton. 2026. DoubleAgents: Human-Agent Alignment in a Socially Embedded Workflow. In *2026 ACM Conference on Human-AI Complementarity and Alignment (HCOMP 2026)*, September 27–30, 2026, Alexandria, VA, USA. ACM, New York, NY, USA, 22 pages. <https://doi.org/10.1145/3834580.3838737>

1 Introduction

Alignment between humans and AI is the foundation of useful agentic systems. To act on a user’s behalf, an agent must understand intent that is often implicit, evolving, and context-dependent — internalizing tacit norms of when (not) to act, when (not) to ask, and how to match a user’s style without being told each time. Without alignment, small misunderstandings scale into systemic problems across workflows and social relationships, eroding the trust that makes delegation possible.

Even well-aligned AI still needs human oversight, both to correct errors and to guide new edge cases in complex, real-world settings. The end goal of alignment is not to fully replace human judgment but to let humans offload as much as possible to AI while maintaining control. How much is offloaded should be up to the human: the more aligned the system, the more confidently a user can delegate. But alignment cannot be specified upfront. Users often do not know what they want until they see the system act, and their preferences evolve as the task unfolds. The central question is *how humans and AI can work together through repeated interaction and refinement to reach alignment and then comfortable offloading*.

Distributed cognition [13, 16] offers a principled framework for designing this kind of human-AI partnership. Research on complex sociotechnical systems has shown that for many hard problems, like flying an airplane, no single individual is fully responsible for the cognition. Instead, cognition is distributed across humans and artifacts: the pilot, co-pilot, autopilot, cockpit instruments, ground control, and maintenance crews all contribute. Three components make this work. First,  *tracking state*: the system must maintain a shared sense of where things stand, as cockpit instruments monitor altitude, speed, and position in real time. Second,  *representing information*: critical state must be displayed in the right form so actors can make judgments at a glance, as cockpit displays show system overviews rather than raw data. Third,  *adapting policies*: all actors follow procedures and checklists that encode accumulated knowledge about safe operation, as aviation checklists evolve after incidents to cover new situations.

We apply this framework to design DoubleAgents, an interactive system for human-agent alignment in socially embedded coordination tasks. The name reflects the system’s two types of AI agents: a *coordination agent* that manages the task on behalf of the user, and *simulated respondent agents* — either real recipients in live deployment or simulated AI agents that stand in for real people

during rehearsal runs. This dual mode lets users iterate on alignment through either live deployment with realistic feedback or simulation that compresses weeks into minutes before real social stakes are involved. Our domain is organizing a university speaker seminar series, a task where a human must monitor evolving situations over weeks, follow up with the nonresponsive, and handle unexpected scenarios, all while navigating real social stakes (you cannot unsend an email, nor be too pushy or too passive).

DoubleAgents distributes cognition through three mechanisms:  a *coordination agent* that tracks all state and proposes next actions, serving as the organizer’s memory and reasoning partner;  a *dashboard* with temporal, social, and procedural views that make the agent’s reasoning legible so the user can verify and evaluate proposals; and  a *policy module* that turns users’ direct edits into reusable alignment artifacts (coordination policies, email templates, and stop hooks) that improve over repeated runs through either live deployment or simulation.

2 Related Works

2.1 Alignment in Human-AI Systems

As AI agents take on more complex tasks, ensuring they align with human values has become a central research challenge. Following the Bidirectional Human-AI Alignment framework [31], we view alignment in agentic settings as an ongoing co-adaptation process. On the system side, agents must be continuously shaped through interaction history and curated context so their actions reflect a user’s goals, preferences, and social norms [8, 43]. On the human side, users must develop appropriate mental models and usage patterns to effectively direct, evaluate, and calibrate delegation [3, 39]. Operationalizing this bidirectional view requires mechanisms that let users specify intent, oversee execution, and revise expectations over time. Terry et al. [37] decomposes this challenge into specification, process, and evaluation alignment, capturing the full arc from goal communication through output verification. Subsequent work has addressed each stage: Shankar et al. [30] shows with EvalGen that users’ evaluation criteria themselves drift as they observe model outputs, while Epperson et al. [7] supports process-level steering with AGDebugger, enabling what-if probing and checkpoint-based resets to diagnose misalignment in multi-agent workflows. Together, these efforts outline a design space where guardrails, prompt optimization, and interactive debugging serve as complementary post-hoc alignment mechanisms.

Constitutional AI offers a different strategy, replacing constant human feedback with rule-based self-supervision using explicit principles [5]. This has been expanded into participatory frameworks such as Collective Constitutional AI, where large groups co-author alignment rules to reflect diverse values [15], and ConstitutionMaker, which helps users turn natural language feedback into usable policies [25]. DoubleAgents contributes to this space by embedding alignment into an interactive process. Following Suchman’s view of plans as situated resources rather than fixed programs [34], policies and stop hooks in our system function as evolving artifacts that users negotiate in context. Through interactions, users iteratively refine coordination policies, email tone, and stop conditions, treating alignment as a continuous, participatory process shaped by the situations in which it unfolds.

2.2 Agentic AI: From Task Planning to Socially Embedded Workflows

Delegating complex tasks to AI agents raises concerns around trust, transparency, and alignment [1], motivating designs that keep humans in the loop. Exposing agent reasoning with user intervention improves both performance and trust [11], yet cognitive research suggests humans rarely plan exhaustively upfront [17, 36], pointing to a need for agents that support incremental, context-sensitive planning with human oversight. Whether agents operate through tool use [29] or direct interface manipulation [4, 18, 20, 21], the difficulty of robust multi-step planning has been consistently highlighted in recent benchmark works [38, 40, 41, 44], underscoring that oversight is not merely desirable but necessary.

These challenges intensify when agents operate in socially embedded contexts, where correctness depends on interpersonal norms and situated judgment rather than functional specifications. Benchmarks for social intelligence reveal a wide capability gap: SOTOPIA [45] found that LLM agents significantly underperform humans on social commonsense, while CASA [27] showed high norm violation rates in agentic settings, suggesting task-directed focus actively crowds out social sensitivity. DoubleAgents targets the gap this literature exposes: the absence of end-to-end systems for multi-turn, socially embedded coordination. Rather than maximizing autonomy, our system helps users iteratively author and refine agent behavior through calibrating delegation boundaries, coordination policies, and communication tone, for tasks where social context and timing are integral to correctness.

2.3 Human Simulations with LLMs

Recent work has shown that LLM-based multi-agent simulations can exhibit human-like behaviors in social and coordination settings, forming routines, organizing events, and interacting in ways that resemble human reasoning patterns [14, 23, 24]. These simulations have been used to approximate human behavior in domains such as economic decision-making [2, 32, 33], judicial reasoning [9], and psychology experiments [12]. They have also supported applications such as strategic gameplay [35], adversarial testing [26], and value alignment through simulated interactions [22].

In DoubleAgents, LLM-simulated respondents are used to provide a fast and flexible way to surface a range of coordination scenarios, including potential edge cases, so that users can iteratively refine their delegation strategies. These interactions primarily serve to accelerate the alignment process by making system behavior observable early, while similar workflows can also be carried out during real-world deployment with actual participants.

3 DoubleAgents System

DoubleAgents is an interactive system for aligning agentic AI behavior with user preferences in coordination tasks. We ground the system in a concrete use case: organizing a university speaker seminar series (see Appendix A for additional background), a setting where users must manage evolving constraints, follow up with nonresponsive participants, and handle unexpected situations under real social stakes. Consistent with our distributed cognition framework, DoubleAgents distributes coordination across three

components: a coordination agent that tracks task state and proposes next plans and actions, a dashboard that presents temporal, social, and procedural views to make the agent’s reasoning legible for user evaluation, and a policy module that turns user edits into reusable alignment artifacts (e.g., coordination policies, email templates, and stop hooks). Together, these components support iterative refinement of agent behavior over time, enabling users to progressively align the system through interaction in both simulated and real-world deployment settings. In this section, we first present a user walkthrough (Section 3.1), followed by the system implementation (Section 3.2).

3.1 User Walkthrough

To illustrate how DoubleAgents operates in practice, we walk through a concrete example. Consider Lee, a PhD student organizing the HCI seminar series at his university. Lee plans to invite four speakers, Prof. Adrian Smith (a famous professor), Prof. Lucy Tran (a rising star professor), Michael Chen (a senior PhD student), and Sarah Malik (a PhD student who is a friend of Lee) to deliver talks during four available time slots: Sep 5, Oct 4, Nov 7, and Dec 5. We demonstrate three example cases of how Lee uses DoubleAgents to solve the problem: (a) starting email bunch, (b) handling edge cases, and (c) following up with non-responsive speakers.

3.1.1 Setting up. Lee begins at the landing page (Figure 5), where he specifies his goal: “I am trying to organize a seminar with four speakers and four slots—help me reach out.” He then provides additional details, including the seminar location and the dates of the four time slots. Next, Lee enters information for each speaker, such as their names and email addresses. When operating in a simulated setting, Lee can optionally include brief persona descriptions to approximate the behavior of the speaker. For example, Prof. Adrian Smith is described as “a famous professor who is very busy and often does not reply to emails,” while Sarah Malik is “a friend who is generally responsive.” In real-world deployment, these personas are not required, as interactions occur with actual participants. Finally, Lee uploads several past emails he has sent to seminar speakers. This allows the system to adapt to his communication style, generating messages that are more casual for friends and more formal for faculty.

With this information, the system could send emails to speakers on Lee’s behalf. But Lee is nervous about this. He wants to make sure the system sends well-written, personalized emails; follows up appropriately if speakers do not respond; nudges people without being pushy; fills all four slots without double-booking; and, critically, does not offend a famous faculty member with an overly casual message. Lee does not yet know how the system will behave, so rather than letting it act in the real world, he starts a simulation that will not send any actual emails. Instead, the simulation lets him see exactly how the system reasons, what plans it proposes, and how it responds to different speaker behaviors, all guided by a set of policies he can inspect and refine. Lee is also shown a list of preset policies in the Policy Panel (Figure 2(Left)). These are natural-language rules that will guide the AI’s reasoning: for example, when to follow up, how to handle conflicts, and when to give up on a non-responsive speaker. Lee reviews them, finds them reasonable, and is now ready to start the simulation.

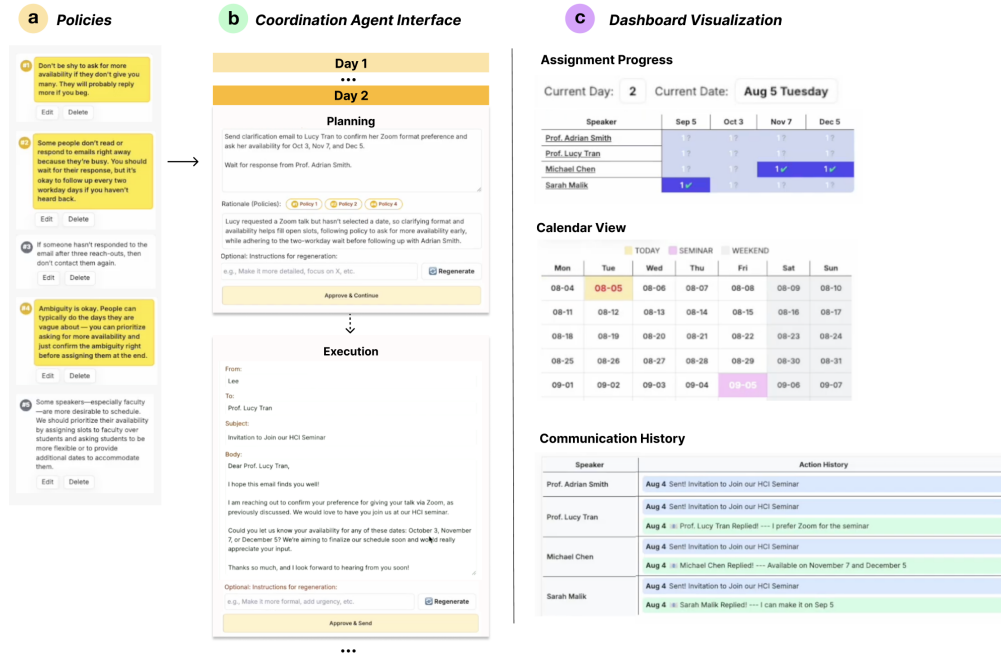


Figure 2: A snapshot of DoubleAgents coordinating four speakers across seminar slots. Grounded in *distributed cognition*, the system organizes interaction across three components: (a) Policies - encoding evolving user preferences and procedures; (b) The Coordination Agent - tracking state, plans next steps, suggests actions, and surfaces the policies guiding its decisions; and (c) The Dashboard Visualization - externalizing key information, including assignments, timelines, and communication history, enabling users to quickly interpret the situation. In use, the coordination agent monitors progress, proposes a plan, and explains its rationale; the user reviews the state through the dashboard, verifies that the plan aligns with expectations, and approves it to proceed. Through interactions, policies are refined, allowing the system to better align with the user over time.

3.1.2 DoubleAgents starts and policy selection. When the simulation starts, Lee sees the Control Panel (Figure 2(Middle)) alongside a dashboard showing the assignment progress, calendar, and communication history (Figure 2(Right)). The calendar shows that in the simulation, today is August 5, and the first seminar date to fill is September 5, so he should start reaching out now. Obviously, nobody has been contacted yet. Lee does not have a clear strategy for who to email first or what to say, so he lets the AI suggest a plan. The coordination agent assesses the current state: no speakers emailed, no availability collected, four slots to fill, and looks for a relevant policy in the existing policy bank, which contains “If someone hasn’t responded to the email after three reach-outs, then don’t contact them again.”, “Don’t be shy to ask for more availability if they don’t give you many. They will probably reply more if you beg.” and a few others. Then the agent finds one that’s most relevant: “If we are just beginning and have not yet contacted any speakers for availability, then ask each speaker for their availability across all slots.” The selected policy is clearly highlighted to Lee in the Policy Panel (Figure 2(Left)), so Lee can see exactly why the AI is about to do what it does. Based on this policy, the AI subsequently proposes a plan (Figure 6a): “Send email to Prof. Adrian Smith, Prof. Lucy Tran, Michael Chen, and Sarah Malik to request availability for slots on Sep 5, Oct 4, Nov 7, and Dec 5.”

This makes sense to Lee: scheduling cannot proceed without first collecting availability. So he clicks ‘Approve & Continue.’ The

system then generates four concrete actions, one email per speaker (Figure 6b), which Lee reviews and approves. Now the AI drafts the actual emails (Figure 6c). Lee reads the first draft carefully. He notices the date format is inconsistent and wants to add a note mentioning that his advisor recommended the speaker. He makes these edits and clicks ‘Regenerate.’ The revised draft incorporates all his changes. For the next email to Prof. Lucy Tran, the system has already learned Lee’s preferred date format and applies it automatically, requiring no further corrections. Lee approves and sends all four emails. Even though these are simulated, Lee is reassured: the AI is learning his style, and he can see exactly what would be sent in the real world.

3.1.3 Handling responses from speakers. In reality, Lee would have to wait days or weeks for responses. In the simulation, time is compressed: a full day in the real world takes only one to two minutes. So Lee quickly sees what happens next. By the end of Day 1, three of the four speakers have responded. Michael Chen and Sarah Malik reply with their availability. Prof. Lucy Tran also replies promptly, but with an unexpected request: “Thanks for reaching out, Lee! I was wondering if there might be an option to present via Zoom instead of in person?” Prof. Adrian Smith, true to his persona, does not respond at all. The assignment progress tracker immediately updates to reflect who has replied and what slots they are available for. The communication log records every exchange. Lee can see at a glance where things stand and where the gaps are.

3.1.4 Edge cases handling. Prof. Lucy Tran’s Zoom request creates a problem: none of the existing policies address remote presentations. The AI does not try to handle this on its own. Instead, the edge case detector flags it and asks Lee for guidance: “Prof. Lucy Tran requests a Zoom option. Should I confirm Zoom availability or insist on in-person participation?” This is exactly the kind of situation Lee was worried about: the AI encountering something unexpected and making a bad call. But instead of acting autonomously, the system pauses and asks. Lee decides that Zoom is acceptable and types: “Zoom is allowed.” The AI generates a response to Prof. Lucy Tran, which Lee reviews and sends. Importantly, Lee’s decision does not resolve this case alone. The system records it as a stop hook and updates the policy set to include a new rule about allowing Zoom when speakers request it. If a similar situation arises later, for example, Michael Chen also asks about Zoom, the system will already know how to handle it without bothering Lee again. At the end of Day 1, the system generates a daily summary of everything that happened: which emails were sent, who replied, and what decisions were made. Lee reviews it, finds it accurate, and clicks ‘Continue to Next Day.’

3.1.5 Clarifying, waiting, and following up. The calendar advances. Prof. Adrian Smith has not yet responded, while Prof. Lucy Tran’s Zoom details remain unresolved. The AI generates a plan: clarify Zoom logistics with Prof. Tran and continue waiting for Prof. Smith (Figure 2(Middle)). Lee checks the rationale, confirms it references the appropriate policies, and approves. The system drafts a follow-up to Prof. Tran, who replies confirming November 7th via Zoom. The dashboard updates accordingly, leaving October 4th as the only open slot.

On Day 3, Prof. Smith has still not responded. The coordination agent, guided by the follow-up policy specifying action after two workdays of silence, drafts a gentle nudge. Lee notices it matches his usual follow-up style: short and to the point, with no unnecessary pressure. This is something the system picked up from uploaded emails and his earlier edits. He approves without changes. Later on, Prof. Smith replies with availability on November 7th, creating a new conflict: three speakers now favor the same date while October 4th remains unfilled. Lee can see the conflict clearly in the assignment progress tracker. From this point onward, the system continues to generate plans grounded in policies, learn from Lee’s past choices, and execute actions adaptively, resolving the scheduling conflict and filling the remaining slot until the seminar assignment is complete.

3.1.6 Usage conclusion. By the time the DoubleAgents usage concludes, Lee has not only produced a tentative speaker-to-slot assignment but has also built up a set of refined policies, customized email templates, and executable stop hooks that align with his actual preferences. He now knows how the AI handles follow-ups, edge cases, and conflicts, because he has seen it happen in a safe and simulated environment. When moving to real-world deployment, such as sending actual emails to speakers, these artifacts directly guide the agent’s actions, reducing uncertainty and enabling more reliable coordination. Over time, this process accumulates into a reusable set of alignment artifacts that can be applied and further developed in future coordination tasks.

3.2 System Implementations

We implemented DoubleAgents as an interactive simulation system built on the ReAct framework [42]. The system architecture comprises four key components: (1) a *Simulation Engine* that drives an iterative planning-execution loop through a coordination agent and LLM-based simulated respondent agents, (2) a *Policy Module* that encodes user values as natural language rules, supports policy selection and refinement through stop hooks, and captures implicit user preferences from edit behavior, (3) an *Edge Case Detector* that identifies situations beyond existing policy coverage and escalates them for user clarification, and (4) a *Dashboard Visualization* built on Flask that provides real-time visual feedback on assignment progress, communication history, and scheduling timelines. Users interact with the system via a control panel that supports plan review, action editing, and natural-language queries. Full implementation details, including prompts and system architecture, are provided in Appendix B.

3.3 Technical Evaluation

DoubleAgents enables iterative task planning and action execution through two core mechanisms: (1) *policy selection*, which steers plan generation toward user-aligned coordination strategies; and (2) *edge case detection*, which identifies ambiguous or potentially problematic situations and escalates them for user intervention. We evaluate both at the component level; end-to-end coordination quality is assessed through the user study (Section 4) and deployments (Section 5). Full experimental setups, including input conditions, test-suite construction, models, and prompt templates, are provided in Appendix C. We report the results in this section.

3.3.1 Policy Selection. Policy selection retrieves the coordination policies relevant to the current problem state (e.g., unfilled slots, unresponsive speakers) to guide the next planning step. We compare four representations of that state — speaker-only, timeslot-only, their combination, and a summary-based abstraction — on a suite of 30 scenarios drawn from real user-study sessions, scoring F1 against the policies users approved during the study (Appendix C).

As shown in Table 1a, the summary-based representation achieves the highest F1, significantly outperforming all three baselines and indicating that a structured, high-level snapshot of task progress yields more accurate policy retrieval than raw context. Notably, the combined condition does not improve over the simpler single-source variants, suggesting that aggregating low-level context without abstraction is insufficient for effective retrieval. We therefore adopt the summary-based representation as the default input to policy selection; the retrieved policies are passed to the coordination agent to guide plan generation in a policy-aligned manner.

3.3.2 Edge Case Detection. Edge case detection determines whether a respondent’s message falls outside the scope of existing policies and should be escalated: for example, a Zoom request when no policy addresses remote presentations. We compare three prompting conditions, namely zero-shot, few-shot without policy context, and few-shot with policy context, on a suite of 100 manually validated scenarios, reporting accuracy (Appendix C).

As shown in Table 1b, few-shot prompting with policy context achieves the highest accuracy, significantly outperforming both

baselines. Access to explicit policy context lets the model reliably judge whether a scenario is already covered or requires escalation, whereas zero-shot prompting struggles to separate routine variation from true edge cases, and few-shot without policy context tends to over-flag benign messages or miss subtle policy gaps. We adopt the policy-aware few-shot approach as the default, surfacing novel or ambiguous situations for timely user intervention while keeping the coordination workflow smooth.

Prompting Tech.	Mean	SD	Prompting Techni.	Mean	SD
Speaker-only	.27	.054	Zero-shot	.79	.021
TimeSlot-only	.11	.038			
TimeSlot + Speaker	.27	.017	Few-shot w/o Policy	.88†	.000
Summary-based	.70***	.035	Few-shot w/ Policy	.93**	.015

(a) Policy Selection (F1 Score). (b) Edge Case Detection (Accuracy)

Table 1: Technical evaluation results. Best scores in bold. Significance via independent two-sample t-test: † $p < 0.05$ over one baseline, ** $p < 0.01$, * $p < 0.001$ over all baselines.**

4 User Study

4.1 Study Design

4.1.1 Overview. To investigate how users and AI can reach alignment through repeated interaction and refinement — and how this alignment enables comfortable offloading — we recruited 10 participants across two sessions over two consecutive days. Each participant completed approximately 2.5 hours total: a 30-minute preparation and setup phase, followed by a 60-minute usage and evaluation session each day. Our goal was to assess whether comfort with delegation evolves over time, how the system’s three distributed cognition components support users in learning what is safe to delegate, and where users still need to maintain control. During each session, participants worked in a simulated environment to schedule a seminar with four dates and four speakers, while the system gradually adapted to participants’ values and preferences through their policies, email templates, and stop hooks. Participants completed two questionnaires. An *AI perception survey* measured alignment expectations, trust, and comfort with automating planning and execution; it was administered before Session 1, after Session 1, and after Session 2. A *DoubleAgents experience survey* captured UX, task load, and perceptions of system features after each session. Together, these instruments allowed us to track how repeated use shifted alignment, delegation boundaries, and the evolving balance between offloading and control. Detailed study procedures and data analysis methods are provided in Appendix D.

4.1.2 Participants. We recruited 10 participants (7 female, 3 male; aged 21–30, $M = 25.7$) from our university community via personal contacts and departmental mailing lists. All reported daily use of generative AI tools, including AI writing assistants (10/10), coding assistants (7/10), and design tools (5/10). Seven described in-depth knowledge of generative AI models, while three reported a good general understanding of key concepts. Participants’ experience with human coordination tasks varied: 2 had moderate to extensive experience, having led team schedules or group planning multiple times; 3 had some experience, having organized small tasks independently; 4 had minimal experience, primarily helping or shadowing others in coordination tasks; and 1 had no prior experience.

4.2 Results

We organize our findings around the central question of how humans and AI reach alignment through repeated interaction and refinement, enabling comfortable offloading. We examine three aspects: whether comfort with delegation evolves over time (§4.2.1), how the three distributed cognition components support participants in learning what is safe to delegate (§4.2.2), and where participants still need to maintain control (§4.2.3).

4.2.1 Did participants become more comfortable delegating control over time?

Participants reported increased comfort delegating both planning and execution tasks to AI. For planning (Figure 3a), comfort ratings rose across all four sub-tasks (letting the AI track and visualize task state, solve scheduling constraints such as time-slot assignments, reason about context to determine next steps, and monitor for edge cases requiring user attention), with averages climbing from roughly 5/7 before deployment to around 6.5/7 afterward. Constraint solving for slot allocation is a comparatively task-specific capability and showed only marginally significant improvement. In comparison, the planning sub-tasks, tracking, monitoring, and flagging shows strong comfort gains over time, consistent with the distributed-cognition affordances of the coordination agent that help organizers offload ongoing awareness work.

For execution (Figure 3b), the pattern split by autonomy level: participants were already fairly comfortable letting the system *draft* emails on their behalf, and comfort rose further (from around 5.7/7 to nearly 6.8/7 across all four email types, each at least marginally significant). By contrast, comfort with the system both *drafting and sending* emails—where the social stakes of an error are higher—started considerably lower and remained so, though it still improved meaningfully (from roughly 3/7 to 4.3/7 across email types). Among the four email types, initial outreach showed strong gains, likely because its content is relatively formulaic and low-variance; follow-up and confirmation emails improved modestly, while clarification emails showed no significant change over the alignment process—an exception we return to in Section 4.2.3.

Participants’ perceived reliance on DoubleAgents, as measured by the *DoubleAgents Experience survey*, increased significantly (4.90/7 to 5.75/7, $p = 0.02^*$). Participants characterized this not as unconditional reliance, but as increased comfort with delegation as the system’s behavior became more predictable and legible — feeling both aligned with how they wanted to work and comfortable letting the system act on their behalf. P2 articulated this arc explicitly: “I would intervene more at first but if it can demonstrate that it’s consistently able to do it correctly, I’m going to let it act on its own on my behalf more autonomously.”

4.2.2 How did participants use and rate the three distributed cognition components?

● **Policies and templates helped participants encode their values, reducing effort over time.** Carrying over user-defined coordination policies strengthened alignment in planning: participants found the AI’s decisions on next steps, progress tracking, and escalation increasingly reflective of their intent. Email-template

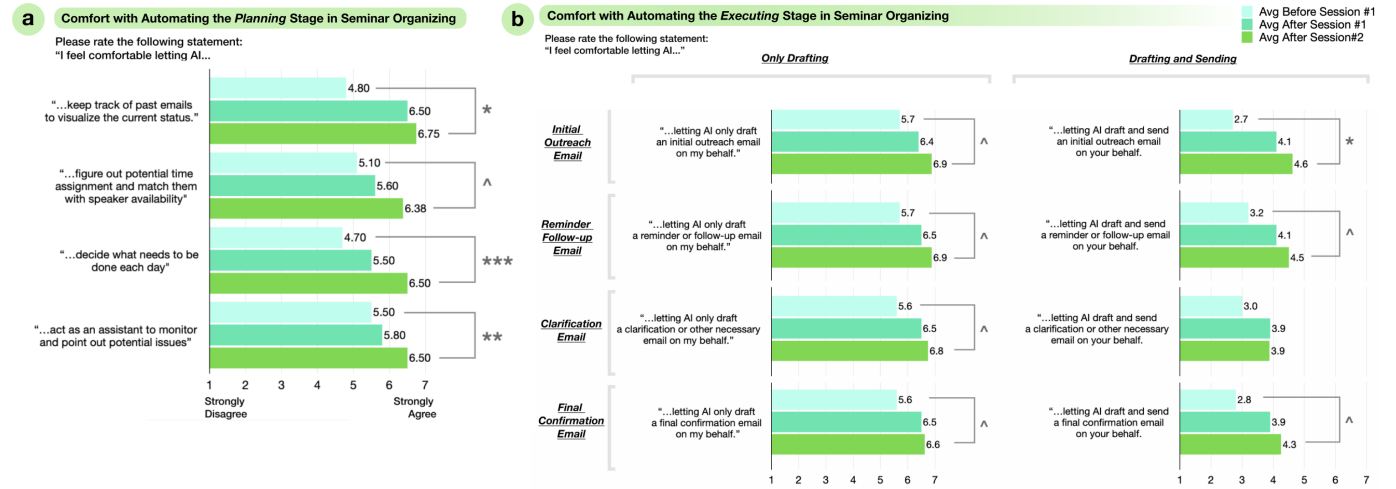


Figure 3: Average user-reported scores across 10 participants for comfort with automating (a) *planning* and (b) *execution* tasks, across sessions over time (1: strongly disagree, 7: strongly agree). [^] $p < 0.1$, * $p < 0.05$, ** $p < 0.01$, *** $p < 0.005$.

carryover similarly improved alignment in execution, with participants reporting that drafts better matched their expectations over successive sessions. Quantitative measures supported this pattern: perceived value alignment via policies and templates increased from 5.50/7 to 6.12/7, drafting comfort rose from approximately 6.0 to nearly 7.0/7 across all email types, and effort dropped from 3.20/7 to 1.88/7 ($p = 0.054^{\wedge}$). P4 noted that edits to greetings, closings, and phrasing carried over into later drafts: "Compared to the last [usage], DoubleAgents get my style... so I edited fewer times this time." P3 observed: "I found that the agent's recent actions followed exactly the policies I had in mind — super aligned."

● **Visualizations made the agent's reasoning legible, enabling confident evaluation.** The dashboard's temporal, social, and procedural views let participants verify what the agent was doing without parsing raw data. P6 found the communication visualization useful for showing full email exchange histories: "Everything is consistent inside the system — when some speakers were not followed up, I could map it back to the visualization and confirm accuracy... [It] helped me build trust... For tomorrow, I'll spend less time checking the email replies [inside the chat interface] one by one, and instead [just check] the Communication History on the side."

● **The coordination agent served as memory and reasoning partner, removing tracking burden.** By maintaining full state across days and weeks of coordination—who was contacted, who responded, what is pending—the agent freed participants from the cognitive overhead of tracking progress themselves. This let them focus on judgment and decision-making rather than bookkeeping. P7 noted that DoubleAgents was "pretty good at figuring out plans, keeping track of who's responded, who's not, who's available, who's not, like what to do moving forward." P1 similarly shared that "now ...I didn't have to use my brain too much [to make sure things are consistent], just a little bit—so it handled the task really well."

4.2.3 Where did participants still need control rather than full delegation? While comfort with delegation grew, participants identified boundaries where human judgment remained essential.

Stop hooks for flagging edge cases. Participants highlighted stop-hook flagging as critical for maintaining safe delegation boundaries. P1 described issue flagging as "one of the most important features of the system": "[With the issue flagging,] I skim [through the other features] now... It made me trust the system because I know it won't just execute something random... It flags when it doesn't know... That helps elicit my values and how I want to handle things — and [then DoubleAgents] respects that..." P8 described a case where flagging prevented a misinterpretation of progress: when they skimmed a reply and thought everything was resolved, the flag showed that it was not. Stop hooks gave participants confidence to offload routine execution precisely because they knew the system would pause at points of uncertainty. Comfort in offloading, in other words, was built not by the agent performing flawlessly, but by making its boundaries of confident action transparent and actionable.

Clarification emails as a natural boundary for delegation. The notable exception to increasing delegation comfort was clarification emails sent without user confirmation, which showed little change across sessions ($p > 0.1$). P10 explained: "For clarification emails, I might want to double-check... [they're] usually not in the plan [or policies]... I'm not comfortable [offloading that] to AI..." Unlike routine emails that follow established templates, clarification emails require the agent to identify ambiguity, judge what information is missing, and formulate the right question — all of which demand user-specific judgment that resists encoding into policies. This suggests a natural boundary for delegation: as agents take on more execution, the critical human role shifts toward decision-making at points of uncertainty, precisely the moments where knowing "when and how to ask the user" becomes the key alignment challenge.

4.2.4 Simulation as an accelerator for alignment. The three alignment mechanisms described above—policies, visualizations, and the coordination agent—improve through repeated interaction. But in live deployment, edge cases surface slowly: a speaker may take days to reply, and unusual requests arise only once across an entire series. Simulation accelerates this by compressing weeks of coordination into minutes while maintaining realistic diversity of cases. Participants found simulations realistic (5.90/7 to 6.75/7), useful for surfacing potential problems (5.80/7 to 6.50/7), and confidence in DoubleAgents robustness increased through simulation (5.80/7 to 6.38/7). P3 described: “*The simulations felt realistic and true to the person [they] were simulating... it elicited possible problems that would happen in real life... Like people asking for funding or needing to reschedule—it forced me to think about how I would handle that and to create relevant policies. I can trust the system to handle that in the future since it already happens in the simulation.*”

Simulation served all three mechanisms as a testbed: it provided a safe environment to probe whether agent behavior matched user intent (*policies*), populated the dashboard with realistic state for practicing oversight (*visualizations*), and surfaced ambiguous situations where the agent should pause (*stop hooks*)—letting participants define delegation boundaries in advance rather than discovering them through live mistakes. This parallels unit testing in software engineering, but where correctness is user-specific rather than universally defined. Simulation is not required—alignment can develop through live use—but it compresses the iteration cycle and lets participants build confidence before real social stakes are involved.

5 Deployment Studies

5.1 Study Design

We conducted three real-world event-planning deployments (D1–D3) with two organizers (Q1, Q2), both PhD students (one male, one female; average age 31) with prior experience organizing academic events and using generative AI for email writing. Table 2 summarizes the deployments.

Deployment ID	D1	D2	D3
Organizer ID	Q1	Q2	Q1
Task	Fall seminar	Workshop panel	Spring seminar
Date	Aug 2025	Sep 2025	Jan–Feb 2026
Candidate pool	23	18	20
Speakers invited	10	6	11
Deployment length	7 days	4 days	12 days

Table 2: Summary of three real-world deployments.

After providing consent, organizers were introduced to DoubleAgents through a simulation-based demonstration of planning–execution loops. Before deployment, each organizer completed at least two simulation sessions (2+ hours) using personas based on real candidates. They refined policies, templates, and contextual inputs until the agent’s proposed actions aligned with their expectations.

For live deployment, we disabled the respondent module and connected DoubleAgents to Gmail’s SMTP service, enabling organizers to manage real email-based coordination tasks. After confirming sufficient alignment, organizers used the system throughout the deployment periods shown in Table 2. The agent operated on live emails while organizers retained approval authority over all actions.

We logged system interactions, including email sends, scheduling updates, and policy adaptations, and collected qualitative feedback through regular check-ins.

5.2 Deployment Results

Both organizers reported that DoubleAgents fit seamlessly into email-heavy coordination workflows that conventional tools cannot manage, attributing their willingness to offload tasks to explicit, editable policies and templates that made the agent’s reasoning legible. Q2 valued editable policies over AI tools that hide reasoning and remarked, “*I trust AI [systems] for specific use cases more.*” Q1 stressed that messages must make speakers “feel they are special” and found DoubleAgents generated strong starter drafts that improved over time. Interactive simulation helped establish alignment: both organizers used real invitee names and persona-specific edge cases to refine policies and templates. While alignment transferred to predictable scenarios, deployment revealed unanticipated dynamics requiring ongoing policy and tone adjustments; Q1 found renewed simulation necessary because “*real life is super complex.*”

DoubleAgents redistributed cognitive labor through three mechanisms: ● **Visualization as shared group representation.** Q1 found the color-coded visualization of speaker states useful enough to adopt a similar scheme for other spreadsheet-based tracking tasks, while Q2 said it supported trust through “*natural mapping*” to his workflow. ● **Policy and artifact refinement as externalized strategy.** Both organizers iteratively edited policies and templates to improve alignment; Q1 adjusted follow-up frequency, formality, and tone, while Q2 valued explicit, editable rationales. These artifacts accumulated organizational knowledge over time, reducing review. ● **Coordination agent as attention manager.** DoubleAgents managed overlapping reply-and-assign cycles, letting Q2 check coordination status without scanning his inbox. Q1 said “*DoubleAgents was really helpful and saved me a lot of time... Policy-abiding [actions and template-following] emails are really useful,*” but emphasized that “*The simulation part cannot be skipped.*”

6 Limitations and Future Work

Sample and study design. Our lab study involved 10 participants in simulated scenarios, limiting generalizability, though the within-subjects design across two sessions yielded rich insights into how alignment strategies and delegation boundaries evolved with repeated use, and three week-long deployments validated these patterns in real-world settings. All participants were frequent generative-AI users; larger, more diverse samples, including users with less AI familiarity, would help clarify onboarding challenges and how simulation supports alignment across experience levels.

Task scope. All of our evaluation is scoped to a single family of coordination tasks: seminar and event scheduling. Every policy, persona, seed rule, deployment, and technical evaluation instance is scheduling-specific, and the unpredictability our simulations surface (e.g., late replies, modality requests, date conflicts) is likewise scheduling-specific. Other socially embedded coordination settings we discuss as motivation in Appendix A, such as shift assignment and interview scheduling, share the same structural features—evolving state, social stakes, and negotiation over availability—

but are not evaluated here. Whether the framework’s three components, and the alignment artifacts users accumulate, transfer beyond scheduling-type coordination remains an open empirical question.

Simulation fidelity. While our simulations reflected realistic coordination challenges, they cannot fully capture real-world unpredictability, and we validated them only through participant-rated realism rather than against real correspondence. Because lab-study personas were hand-authored by the research team, policies refined in simulation may partly reflect our assumptions about how invitees behave; deployment organizers mitigated this by building personas from real candidates. Grounding both the coordination agent and the respondent simulator in users’ historical data — past emails, decisions, and communication patterns — could enable more personalized modeling of values and tone, and validating simulated replies against real ones is a natural next step.

Model dependence. DoubleAgents assigns different models to different subtasks (Appendix B.5): a reasoning-optimized model for coordination planning, where deliberation quality matters most, and a faster general-purpose model for drafting, simulation, and edge-case detection, where per-call latency and cost dominate because these run many times per simulated day. Our technical evaluation varied state representation and shot count but not prompt phrasing or length, so sensitivity to prompt formatting is untested. Future models with stronger social reasoning and long-horizon planning may improve coordination quality, and tighter integration between user-specific data and model reasoning remains an open challenge.

7 Conclusion

We present DoubleAgents, an interactive alignment tool for human coordination through user intervention, value-reflecting policies, rich state visualizations, and uncertainty flagging. Technical evaluations validate key system capabilities of DoubleAgents, including policy selection and edge-case detection. A two-session user study (N=10) shows users’ increased willingness to delegate both planning and execution, supported by policy-based alignment, interpretable visualizations, and explicit delegation boundaries. Three real-world deployments show that alignment learned through simulation transfers to live coordination while continuing to evolve in practice. Our findings frame alignment not as a one-time specification problem but as an iterative process, and position distributed cognition as a principled approach for enabling progressive delegation with retained user control.

Acknowledgments

This work was supported by the Columbia Center for AI and Responsible Financial Innovation (CAIRFI). We would like to thank people from the Data, Agents, and Processes Lab (DAPLab) at Columbia University for their valuable discussions on the paper. We also thank all reviewers for their constructive feedback and suggestions, which significantly improve our work.

References

- [1] Saleh Afroogh, Ali Akbari, Emmie Malone, Mohammadali Kargar, and Hananeh Alambeigi. 2024. Trust in AI: Progress, Challenges, and Future Directions. *Humanities and Social Sciences Communications* 11 (2024), 1568. doi:10.1057/s41599-024-04044-8

- [2] Gati Aher, Rosa I. Arriaga, and Adam Tauman Kalai. 2023. Using Large Language Models to Simulate Multiple Humans and Replicate Human Subject Studies. arXiv:2208.10264 [cs.CL] <https://arxiv.org/abs/2208.10264>
- [3] Robert W. Andrews, J. Mason Lilly, Divya Srivastava, and Karen M. Feigh. 2023. The Role of Shared Mental Models in Human-AI Teams: A Theoretical Review. *Theoretical Issues in Ergonomics Science* 24, 2 (2023), 129–175. doi:10.1080/1463922X.2022.2061080
- [4] Anthropic. 2025. Introducing Computer Use, a New Claude 3.5 Sonnet, and Claude 3.5 Haiku. *Anthropic News* (March 2025). <https://www.anthropic.com/news/3-5-models-and-computer-use>
- [5] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, et al. 2022. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073* (2022).
- [6] Jeffrey Basoah, Daniel Chechelnitsky, Tao Long, Katharina Reinecke, Chrysoula Zerva, Kaitlyn Zhou, Mark Diaz, and Maarten Sap. 2025. Not Like Us, Hunty: Measuring Perceptions and Behavioral Effects of Minoritized Anthropomorphic Cues in LLMs. In *Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency (FAccT ’25)*. Association for Computing Machinery, New York, NY, USA, 710–745. doi:10.1145/3715275.3732045
- [7] Will Epperson, Gagan Bansal, Victor C Dibia, Adam Fourney, Jack Gerrits, Erkan Zhu, and Saleema Amershi. 2025. Interactive debugging and steering of multi-agent ai systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–15.
- [8] Ge Gao, Alexey Taymanov, Eduardo Salinas, Paul Mineiro, and Dipendra Misra. 2024. Aligning LLM Agents by Learning Latent Preference from User Edits. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. 136873–136896. doi:10.52202/079017-4349
- [9] Sil Hamilton. 2023. Blind Judgement: Agent-Based Supreme Court Modelling With GPT. arXiv:2301.05327 [cs.CL] <https://arxiv.org/abs/2301.05327>
- [10] Sandra G Hart. 2006. NASA-task load index (NASA-TLX); 20 years later. In *Proceedings of the human factors and ergonomics society annual meeting*, Vol. 50. Sage publications Sage CA: Los Angeles, CA, 904–908.
- [11] Gaole He, Gianluca Demartini, and Ujwal Gadiraju. 2025. Plan-Then-Execute: An Empirical Study of User Trust and Team Performance When Using LLM Agents As A Daily Assistant. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM. doi:10.1145/3706598.3713218
- [12] Luke Hewitt, Ashwini Ashokkumar, Isaias Ghezae, and Robb Willer. 2024. Predicting Results of Social Science Experiments Using Large Language Models. (2024). Working Paper.
- [13] James Hollan, Edwin Hutchins, and David Kirsh. 2000. Distributed cognition: toward a new foundation for human-computer interaction research. *ACM Trans. Comput.-Hum. Interact.* 7, 2 (June 2000), 174–196. doi:10.1145/353485.353487
- [14] John J. Horton. 2023. Large Language Models as Simulated Economic Agents: What Can We Learn from Homo Silicus? arXiv:2301.07543 [econ.GN] <https://arxiv.org/abs/2301.07543>
- [15] Saffron Huang, Divya Siddarth, Liane Lovitt, Thomas I. Liao, Esin Durmus, Alex Tamkin, and Deep Ganguli. 2024. Collective Constitutional AI: Aligning a Language Model with Public Input. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT ’24)*. Association for Computing Machinery. doi:10.1145/3630106.3658979
- [16] Edwin Hutchins and Tove Klausen. 1996. Distributed cognition in an airline cockpit. *Cognition and communication at work* 1 (1996), 15–34.
- [17] Michel Krieger, Emily Margarette Stark, and Scott R Klemmer. 2009. Coordinating tasks on the commons: designing for personal goals, expertise and serendipity. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 1485–1494.
- [18] Amazon AGI Labs. 2025. Introducing Amazon Nova Act. *Amazon Science Blog* (March 2025). <https://labs.amazon.science/blog/nova-act>
- [19] Tao Long, Katy Ilonka Gero, and Lydia B Chilton. 2024. Not Just Novelty: A Longitudinal Study on Utility and Customization of an AI Workflow. In *Proceedings of the 2024 ACM Designing Interactive Systems Conference (Copenhagen, Denmark) (DIS ’24)*. Association for Computing Machinery, New York, NY, USA, 782–803. doi:10.1145/3643834.3661587
- [20] Manus AI. 2025. Manus: The General AI Agent. <https://manus.im/> Accessed: 2025-04-08.
- [21] OpenAI. 2025. Introducing Operator. *OpenAI Blog* (January 2025). <https://openai.com/index/introducing-operator/>
- [22] Xianghe Pang, Shuo Tang, Rui Ye, Yuxin Xiong, Bolun Zhang, Yanfeng Wang, and Siheng Chen. 2024. Self-Alignment of Large Language Models via Monopolylogue-based Social Scene Simulation. *arXiv preprint arXiv:2402.05699* (2024).
- [23] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST ’23)*. ACM, X–X.
- [24] Joon Sung Park, Lindsay Popowski, Carrie Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2022. Social simulacra: Creating populated prototypes for social computing systems. In *Proceedings of the 35th Annual ACM*

- Symposium on User Interface Software and Technology*. 1–18.
- [25] Savvas Petridis, Benjamin D. Wedin, James Wexler, Mahima Pushkarna, Aaron Donsbach, Nitesh Goyal, Carrie J. Cai, and Michael Terry. 2024. Constitution-Maker: Iteratively Critiquing Large Language Models by Converting Feedback into Principles. In *Proceedings of the 29th International Conference on Intelligent User Interfaces (IUI '24)*. ACM, 853–868.
- [26] Alberto Purpura, Sahil Wadhwa, Jesse Zymet, Akshay Gupta, Andy Luo, Melissa Kazemi Rad, Swapnil Shinde, and Mohammad Shahed Sorower. 2025. Building Safe GenAI Applications: An End-to-End Overview of Red Teaming for Large Language Models. arXiv:2503.01742 [cs.CL]. <https://arxiv.org/abs/2503.01742>
- [27] Haoyi Qiu, Alexander Fabbri, Divyansh Agarwal, Kung-Hsiang Huang, Sarah Tan, Nanyun Peng, and Dongyeop Wu. 2025. Evaluating Cultural and Social Awareness of LLM Web Agents. In *Findings of NAACL*.
- [28] Steven I. Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D. Weisz. 2023. The Programmer’s Assistant: Conversational Interaction with a Large Language Model for Software Development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces (Sydney, NSW, Australia) (IUI '23)*. Association for Computing Machinery, New York, NY, USA, 491–514. doi:10.1145/3581641.3584037
- [29] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: language models can teach themselves to use tools. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 2997, 13 pages.
- [30] Shreya Shankar, JD Zamfirescu-Pereira, Björn Hartmann, Aditya Parameswaran, and Ian Arawjo. 2024. Who validates the validators? aligning llm-assisted evaluation of llm outputs with human preferences. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. 1–14.
- [31] Hua Shen, Tiffany Kneare, Reshmi Ghosh, Kenan Alkiek, Kundan Krishna, Yachuan Liu, Ziqiao Ma, Savvas Petridis, Yi-Hao Peng, Li Qiwei, Sushrita Rakshit, Chenglei Si, Yutong Xie, Jeffrey P. Bigham, Frank Bentley, Joyce Chai, Zachary Lipton, Qiaozhu Mei, Rada Mihalcea, Michael Terry, Diyi Yang, Meredith Ringel Morris, Paul Resnick, and David Jurgens. 2025. Position: Towards Bidirectional Human-AI Alignment. doi:10.48550/arXiv.2406.09264 arXiv:2406.09264 [cs].
- [32] Karthik Sreedhar, Alice Cai, Jenny Ma, Jeffrey V Nickerson, and Lydia B Chilton. 2025. Simulating Cooperative Prosocial Behavior with Multi-Agent LLMs: Evidence and Mechanisms for AI Agents to Inform Policy Decisions. In *Proceedings of the 30th International Conference on Intelligent User Interfaces (IUI '25)*. Association for Computing Machinery, New York, NY, USA, 1272–1286. doi:10.1145/3708359.3712149
- [33] Karthik Sreedhar and Lydia B. Chilton. 2025. Simulating Human Strategic Behavior: Comparing Single and Multi-agent LLMs. In *Proceedings of the 58th Hawaii International Conference on System Sciences (HICSS)*.
- [34] Lucille Alice Suchman. 1987. *Plans and situated actions: The problem of human-machine communication*. Cambridge university press.
- [35] Meta Fundamental AI Research Diplomacy Team, Anton Bakhtin, Noam Brown, Emily Dinan, Gabriele Farina, et al. 2022. Human-level play in the game of Diplomacy by combining language models with strategic reasoning. *Science* 378, 6622 (2022), 1067–1074. doi:10.1126/science.ade9097
- [36] Jaime Teevan, Shamsi T Iqbal, Carrie J Cai, Jeffrey P Bigham, Michael S Bernstein, and Elizabeth M Gerber. 2016. Productivity decomposed: Getting big things done with little microtasks. In *Proceedings of the 2016 CHI Conference Extended Abstracts on Human Factors in Computing Systems*. 3500–3507.
- [37] Michael Terry, Chinmay Kulkarni, Martin Wattenberg, Lucas Dixon, and Meredith Ringel Morris. 2023. Interactive AI alignment: specification, process, and evaluation alignment. arXiv preprint arXiv:2311.00710 (2023).
- [38] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. arXiv preprint arXiv:2407.18901 (2024).
- [39] Lindsey Morgan Vanderlyn, Dirk Vath, and Thang Vu. 2025. Understanding the Role of Mental Models in User Interaction with an Adaptive Dialog Agent. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, 989–1015. doi:10.18653/v1/2025.findings-naacl.56
- [40] Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. 2024. Travelplanner: A benchmark for real-world planning with language agents. arXiv preprint arXiv:2402.01622 (2024).
- [41] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. (2024). arXiv:2406.12045 [cs.AI]. <https://arxiv.org/abs/2406.12045>
- [42] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.
- [43] Shuning Zhang, Ying Ma, Jingrui Chen, Simin Li, Xin Yi, and Hewu Li. 2025. Towards Aligning Personalized Conversational Recommendation Agents with Users’ Privacy Preferences. arXiv preprint arXiv:2508.07672 (2025). <https://arxiv.org/abs/2508.07672>
- [44] Huaixiu Steven Zheng, Swaroop Mishra, Hugh Zhang, Xinyun Chen, Minmin Chen, Azade Nova, Le Hou, Heng-Tze Cheng, Quoc V Le, Ed H Chi, et al. 2024. Natural plan: Benchmarking llms on natural language planning. arXiv preprint arXiv:2406.04520 (2024).
- [45] Xuhui Zhou, Hao Zhu, Leena Mathur, Ruohong Zhang, Haofei Qi, Zhengyang Yu, Louis-Philippe Morency, Yonatan Bisk, Daniel Fried, Graham Neubig, and Maarten Sap. 2024. SOTOPIA: Interactive Evaluation for Social Intelligence in Language Agents. In *ICLR*.

A Seminar Planning: A Human Coordination Problem

Human coordination—such as assigning work shifts or scheduling job interviews—is both cognitively demanding and socially delicate. Unlike tasks with stable inputs and predictable outcomes, coordination requires managing uncertainty, reconciling multiple perspectives, and sustaining relationships over time. Even small lapses—missed follow-ups, unclear tone, or misaligned expectations—derail coordination and strain relationships. These challenges make coordination both essential and persistently difficult in many areas of knowledge work.

We study these challenges through one concrete and consequential setting: organizing an academic seminar series. A typical series spans an entire semester and involves inviting a new external speaker nearly every week, amounting to 8–12 speakers per term. Early on, organizers must identify potential speakers, gather their availability constraints, and balance their own preferences—such as ensuring topical diversity across weeks and securing highly sought-after senior researchers who are harder to schedule.

However, the most unpredictable and challenging part is dealing with responses. Some people do not reply at all and require multiple follow-ups to elicit a response. Some give only vague and tentative replies that need repeated confirmation. Some can be flaky and unreliable, offering slots but later canceling. And many unexpected problem cases arise: people might be on parental leave, ask for visa letters or funding support, recommend a different speaker, request a different date or talk modality, or raise logistical demands. In such human coordination tasks, you can rarely predict what is coming. Therefore, conventional scheduling tools such as Calendly¹ or When2Meet² are ill-suited to this setting. They assume stable constraints and one-shot coordination, yet seminar scheduling requires ongoing negotiation, persuasion, and relationship maintenance. Unlike ordinary meeting scheduling, seminar speakers are doing you a favor, which means you cannot simply send them a link to pick a time—you must persuade and negotiate. Social dynamics and etiquette also play a large role: organizers must time their emails carefully to avoid overwhelming, annoying, or losing momentum with potential speakers, who are often busy scholars. Knowing when to nudge, follow up, or wait is subtle. Email remains the standard medium for professional and academic outreach, allowing organizers to adapt tone—formal, casual, or enthusiastic—to different speakers in ways scheduling links cannot. It also supports sharing rich context, such as talk topics, background, past speakers, or travel plans—far beyond simply choosing a calendar slot.

Thus, experienced organizers adopt personal strategies to cope with such a demanding task. For example, some follow a rule of

¹<https://calendly.com/>

²<https://www.when2meet.com/>

“wait two days, then follow up” to nudge busy people who may not read emails right away, while others wait a full week before reaching out again. Some prefer contacting close collaborators or junior researchers first, while others prioritize securing prominent professors or distant speakers early to reduce travel constraints. When conflicts arise between speakers, organizers might “give faculty priority and ask students to be more flexible,” carefully deciding whom to approach for alternative availability to minimize disruption. For non-responders or last-minute cancellations, they must judge when and how often to follow up before moving on to new options—and sometimes even decide when it is appropriate to “beg” busy invitees for additional availability. Because plans are constantly evolving, organizers become locked in a cognitively and time-intensive loop of tracking shifting states, replanning, and executing revised plans as new information arrives—yet they often cannot proceed with the next steps or finalize allocations until replies are received. Since seminar scheduling requires nuanced, context-dependent decisions that reflect each organizer’s values and preferences, it is difficult to fully automate. An effective seminar scheduling tool should not attempt to handle the process independently; instead, it must learn from and adapt to the organizer, respecting social norms, relational dynamics, and the inherent unpredictability of responses.

B System Implementation Details

We present DoubleAgents, a system for aligning agentic AI through interactive simulation. Built on the ReAct framework [42], DoubleAgents supports iterative planning and execution via a team of AI agents that collaborate with users on complex coordination tasks. DoubleAgents takes as input a user-defined goal (e.g., “Organize a seminar with four speakers and four slots”), persona descriptions for simulated respondent agents, the user’s existing email templates, and a set of generic scheduling policies. Through iterative simulations, the system produces a finalized assignment of speakers to slots, along with refined policies, updated email templates, and executable stop hooks that users can later deploy in real-world agentic workflows.

The system architecture comprises four key components: (1) a *Simulation Engine* that drives the iterative planning-execution loop through coordinated AI agents (Section B.1), (2) a *Policy Module* that governs agent reasoning and adapts to user preferences (Section B.2), (3) an *Edge Case Detector* that identifies situations beyond existing policy coverage and escalates them for user intervention (Section B.3), and (4) a *Dashboard Visualization* that provides real-time visual feedback on task state and progress (Section B.4).

These four components interact in a loop. The *Policy Module* supplies policies to the coordination agent inside the *Simulation Engine*, which in turn exchanges emails with simulated respondent agents to advance the scheduling task. When a respondent’s reply falls outside existing policy coverage, the *Edge Case Detector* escalates the issue to the user and feeds the resulting clarification back into the *Policy Module* as an updated policy. Throughout this cycle, the *Dashboard Visualization* renders the evolving task state so that the user can monitor progress and intervene at any point. We describe each component in detail below.

B.1 Simulation Engine

The Simulation Engine is the core of DoubleAgents, implementing the iterative planning-execution loop that drives the coordination process. It comprises three main subcomponents: the *Coordination Agent*, which reasons over the task and orchestrates actions; the *Simulated Respondent Agents*, which emulate human behavior to create a realistic environment; and the *Context Management Module*, which maintains the evolving problem state.

B.1.1 Coordination Agent. The coordination agent is the primary reasoning component of the system, responsible for assessing the current problem state, selecting relevant policies, generating plans, and executing actions. It operates within the ReAct framework and proceeds through the following stages at each simulation step:

- **Progress Understanding.** The agent reviews the current problem context, including speaker information, email logs from previous days, and the history of user-approved plans and actions, and generates a concise summary of the current status. This summary indicates which days have been scheduled, what actions have been taken, and the most recent contact date for each speaker. The prompt used for generating the progress summary is provided in Appendix E.1.1.
- **Policy Selection.** Based on the progress summary, the agent invokes the policy selection tool to identify the most relevant policies for the current state (Section B.2).
- **Plan Generation.** Guided by the selected policies, the agent proposes a high-level plan (e.g., “Send email to all speakers to request availability for all seminar slots”). The plan and its rationale are presented to the user via the Interactive Interface (Figure 2(Middle)) for review and approval. Users may approve, edit, or request regeneration of the plan (Figure 6a).
- **Action Generation.** Upon plan approval, the agent generates a set of concrete actions to operationalize the plan (Figure 6b). There are three types of actions: (a) drafting emails to selected respondents, (b) sending emails to specified recipients, and (c) awaiting responses from certain respondents. Users retain full control over actions and may edit, remove, or request regeneration. Detailed prompts for plan and action generation are provided in Appendix E.1.3.
- **Action Execution.** The agent instantiates each approved action by invoking the appropriate tools from its toolbox (Figure 4). For email drafting, it calls the `writeEmail()` API, which generates emails given the current problem state, seminar details, user-provided templates, and historical email edits. Email drafting is performed using GPT-4o. For sending, it calls the `sendEmail()` API targeting the appropriate speaker email addresses. For delayed responses, it calls the `waitResponse()` tool, pausing execution and advancing to the next pending action or simulated day.

Users interact with the coordination agent through the Interactive Interface (Figure 2(Middle)), which displays the full history of plans, actions, and emails as interactive pop-up windows. The interface also supports natural language queries (e.g., “What did Prof. Lucy Tran reply in the last email?”), powered by GPT-4o (see Appendix E.4). We use GPT-o3 as the backbone of the coordination agent, given its strong reasoning capabilities.

B.1.2 Simulated Respondent Agents. Drawing inspiration from [23], we employ LLMs to simulate each respondent agent, generating actions that align with their assigned personas. Upon receiving emails from the coordination agent, each simulated respondent first reasons over its persona and then generates an appropriate action. For instance, a respondent whose persona specifies “*You are very busy and reply very late. Normally, it takes you three follow-up emails to respond*” will delay its reply accordingly, while a respondent described as prompt in communication will reply immediately. If a respondent’s persona includes specific preferences (e.g., “prefers presenting over Zoom”), the generated response will reflect those preferences. Each respondent composes and sends email responses, which advance the scheduling process and enable the user to coordinate subsequent steps. The prompt for the speaker agent is detailed in Appendix E.3. We adopt GPT-4o for email response generation via the `sendEmail()` tool.

B.1.3 Context Management Module. The Context Management Module maintains all relevant contextual information throughout the scheduling process. `DoubleAgents` represents the problem context using JSON structures, where each key denotes a specific type of contextual information (e.g., speaker profiles, time constraints, email history) and each value contains the corresponding content. As shown in Figure 4, the problem context includes keys such as “time info” (reflecting seminar slots), speaker details, and communication logs. The context is dynamically updated at each step, incorporating user feedback, agent actions, and simulated responses to maintain an up-to-date representation of the task state.

B.2 Policy Module

The Policy Module serves as the central mechanism through which user values and preferences are encoded and operationalized within the system. Policies are natural language rules that guide the coordination agent’s reasoning and planning at each step of the scheduling process. They are displayed in the Policy Panel (Figure 2(Left)), where users can inspect, add, edit, or delete them at any time.

B.2.1 Seed Policies. To bootstrap the system with broadly applicable guidelines, the lead authors designed a set of six initial “seed policies” that address common scheduling challenges: (1) individuals being reluctant to share their full availability, (2) ambiguous or unclear responses, (3) the need for multiple follow-ups to elicit responses, (4) resolving scheduling conflicts between two speakers, (5) initiating the process by broadly reaching out to potential participants, and (6) determining when to cease follow-up attempts with non-responsive participants. Each policy is designed to address a specific challenge and serves as a default guideline that the coordination agent can draw upon when generating plans.

B.2.2 Policy Selection. At each planning step, the coordination agent invokes a policy selection tool that identifies the most relevant policies given the current problem state. For instance, at the beginning of the scheduling process when no speaker availabilities have been collected, the system selects the policy: “*If we are just beginning and have not yet contacted any speakers for availability, then ask each speaker for their availability across all slots.*” The selected policy is highlighted in the Policy Panel (Figure 2(A)), providing users with a transparent rationale for the agent’s subsequent plan.

Details of the prompts used for policy selection are provided in the Appendix E.1.2.

B.2.3 Policy Refinement via Stop Hooks. Policies are not static; they evolve throughout the simulation process. When the Edge Case Detector (Section B.3) identifies a situation not covered by existing policies and the user provides a clarification, the system records this intervention as a *stop hook* and uses it to refine the policy set. For example, if a speaker requests to present via Zoom and the user specifies a preference for in-person attendance, this clarification is converted into a new policy explicitly stating that remote participation is not preferred. In this way, user interventions during edge cases are transformed into reusable system policies that persist across subsequent simulation steps and can be deployed in real-world workflows.

B.2.4 Implicit User Preference Learning. Beyond explicit policy refinement, the system also supports implicit learning of user preferences by observing user-initiated edits to system-generated plans, actions, or emails. The coordination agent summarizes each edit into a concise statement, logs it in a user edit history, and incorporates it into the problem context for subsequent planning steps. This enables the agent to evolve its behavior in alignment with user preferences over time without requiring explicit policy authoring. We leverage GPT-4o for the summarization process, with prompt details provided in Appendix E.1.5.

B.3 Edge Case Detector

The Edge Case Detector serves as a safeguard that identifies situations falling outside the coverage of existing policies and escalates them to the user for clarification. It is implemented as a tool that the coordination agent invokes to check whether an incoming message from a respondent agent can be addressed using the current policy set.

When the detector identifies an edge case—for example, a speaker requesting to present via Zoom when no existing policy addresses remote participation—it triggers a clarification request through the Interactive Chat Interface. The system presents the user with a concise description of the situation and a question prompting guidance (e.g., “*Prof. Lucy Tran requests a Zoom option due to circumstances preventing in-person attendance. Should I confirm Zoom availability for their seminar or insist on in-person participation?*”). The user then provides a natural language instruction (e.g., “*Tell Prof. Lucy Tran we prefer in-person due to the potentially large audience*”), which the coordination agent processes to generate an appropriate action.

Critically, each user clarification is recorded as a *stop hook* and used to refine the policy set (Section B.2). This mechanism ensures that (1) ambiguous or novel situations are never resolved autonomously without user input, (2) user interventions are preserved as reusable policies for future reference, and (3) the policy set grows iteratively to cover an expanding range of scenarios. The edge case detector is implemented with GPT-4o, with prompts detailed in Appendix E.2.

B.4 Dashboard Visualization

To support user understanding and foster alignment, `DoubleAgents` provides real-time visual feedback on the task state and system

Component	Model	Sec.
<i>Deployed system</i>		
Coordination agent (planning, actions)	o3	B.1.1
Email drafting (<code>writeEmail()</code>)	GPT-4o	B.1.1
Simulated respondents (<code>sendEmail()</code>)	GPT-4o	B.1.2
Free-form QA chat	GPT-4o	E.4
Implicit preference learning	GPT-4o	B.2.4
Edge case detector	GPT-4o	B.3
Daily summary	GPT-4o	B.4
<i>Technical evaluation only</i>		
Policy selection	o4-mini	C.1
Edge case detection	GPT-4o	C.2
Edge case test-suite generation	GPT-5	C.2

Table 3: Model assignment by component. Models in the lower block are used only for the technical evaluation (Appendix C) and are not part of the deployed system.

progress throughout the scheduling process. We implement a Flask-based dashboard interface designed to make the planning workflow interactive, legible, and actionable. The dashboard is composed of several layered visual components, shown in Figure 2 (Right):

- **Assignment Progress Tracker.** This panel visualizes tentative and confirmed speaker-to-slot assignments. It highlights which seminar slots remain unfilled and which speakers have been successfully scheduled, enabling users to assess how close the current plan is to completion at a glance.
- **Calendar View.** Anchored in simulated days, this temporal layer helps users reason about timing, deadlines, and follow-up actions. The calendar highlights both the current simulated day and the seminar dates, allowing prioritization of actions based on urgency and scheduling constraints.
- **Communication History.** This panel displays the message history between the coordination agent and each speaker, offering a transparent view of the outreach and response status. It enables users to track ongoing communications, detect gaps, and assess follow-up needs.

These visual layers operate in parallel, allowing the system to efficiently track and update the state of each element in real time. At the end of each simulated day, the system generates a daily summary that highlights key events and decisions, using GPT-4o (with prompt details in Appendix E.5). The summary is presented within the Interactive Interface, providing users with a consolidated overview before advancing to the next simulated day.

B.5 Model Configuration

Table 3 summarizes the model used for each component. We select models by task demand: coordination planning requires multi-step reasoning over accumulated state and uses GPT-o3, whereas email drafting, respondent simulation, and edge-case detection are shorter-horizon generation and classification tasks that run several times per simulated day, making a faster and cheaper model preferable.

C Technical Evaluation Setup

This appendix details the experimental setup for the two technical evaluations reported in Section 3.3.

C.1 Policy Selection

Policy selection refers to identifying coordination policies that guide the generation of high-quality, user-aligned plans. Given a particular problem state (e.g., unfilled time slots and unresponsive speakers), the policy selection module retrieves relevant policies that can inform the next planning step. For example, if no speakers have been contacted yet, a suitable policy would be: *“If we are just beginning and have not yet contacted any speakers for availability, then ask each speaker for their availability across all slots.”* In our implementation, we adopt a seed set of general-purpose coordination policies curated by domain experts.

To study how different representations of the problem state affect policy selection, we compare four input conditions that vary in the level of abstraction and context provided to the model:

- **Speaker-only** context, which includes information about each speaker’s persona. For example: *“Speaker 1 (Prof. Adrian Smith): busy, often replies only after multiple follow-ups; Speaker 2 (Dr. Lucy Tran): usually responds within 1–2 days; Speaker 3 (Sarah Malik): highly responsive and flexible; Speaker 4 (Michael Chen): prefers early scheduling but has not replied recently.”*
- **TimeSlot-only** context, which includes the current status of all time slots and their constraints. For example: *“Four slots: Sep 5 (unfilled), Oct 3 (filled), Nov 7 (filled), Dec 5 (unfilled). The Nov 7 slot has been confirmed with a speaker, while Oct 3 is tentatively held. Remaining open slots require confirmation.”*
- **Combined TimeSlot and Speaker** context, which includes both sources of information without additional abstraction. For example: *“Speaker 1 (busy, slow to reply) and Speaker 2 (moderately responsive) were last contacted two days ago; Speaker 3 (responsive) has not yet been contacted; Speaker 4 prefers early slots but has not replied. Slot status: Oct 3 and Nov 7 are filled (Nov 7 confirmed), while Sep 5 and Dec 5 remain open.”*
- **Summary-based** context, which includes the information of the speakers, time slots, and the summary of the current task state to provide a structured, high-level snapshot of the system. An example summary reads: *“One slot has been secured: speaker 4 confirmed for Nov 7 (2025-11-07) and received a confirmation email on 2025-08-07. All other speakers are still pending. Last outreach dates: speaker 1 and speaker 2 were last emailed on 2025-08-08; speaker 3 was last emailed on 2025-08-07.”*

Prompt templates for all conditions are provided in Appendix F. We construct a test suite of 30 examples sourced from real user study sessions, covering a diverse set of scheduling scenarios. For each example, we treat the set of policies approved by users during the study as ground truth. All evaluations are conducted using the o4-mini model with a default temperature of 1, and we report the average F1 score across three independent runs for each condition.

C.2 Edge Case Detection

Edge case detection aims to determine whether a respondent’s message introduces a scenario that falls outside the scope of existing coordination policies. For instance, if a speaker requests to present via Zoom while no policy addresses virtual presentations, the system should flag this case for user clarification and further instruction.

To study how different prompting strategies affect detection performance, we compare three conditions that vary in the level of guidance and policy awareness provided to the model: (1) *zero-shot prompting*, where the model is given only the task instruction and the incoming message without any examples; (2) *few-shot prompting without policy context (few-shot w/o policy)*, where the model is provided with representative examples of edge and non-edge cases but does not have access to the current set of coordination policies; and (3) *few-shot prompting with policy context (few-shot w/ policy)*, where the model is given both demonstration examples and the existing policies, allowing it to assess whether a new situation is already covered or requires escalation.

Prompt templates for all conditions are provided in Appendix F.2. We construct a test suite of 100 examples, manually validated by the lead authors to ensure diversity and coverage across common coordination scenarios and edge cases. The construction methodology is detailed in Appendix F.2. All experiments are conducted using GPT-4o, with temperature set to 0.7 and a maximum token limit of 100. We report the average accuracy across three independent runs for each condition.

D User Study Procedures and Data Analysis

Test Scenario Preparation. We selected eight distinct personas identified by three organizing experts, representing realistic and diverse roles, research areas, and backgrounds (e.g., a fifth-year PhD student studying VR currently on the job market; a busy junior professor in NLP who spends much of her time on her startup). For each persona, we specified reply behaviors (responsiveness, email writing style; e.g., slow responder, writes only short emails) and optional issues (potential concerns they might raise with speakers; e.g., requesting additional funding even though their in-person visit is already funded). We split them into two groups and created seminar organizing instances. We set the timeline as a monthly seminar schedule, with the first seminar one month after the simulation start date.

Day 1, Preparation and Setup (30min). Participants first provided informed consent and completed an AI comfortableness survey assessing their perceptions and expectations with AI automation tools, human coordination tasks, and demographics. They also edited a few seeded templated seminars organizing emails according to their tone to feed the system their writing style. They were then introduced to the DoubleAgents interface and task. One researcher gave a brief walkthrough using a visual diagram and demonstration, covering the system’s core components, including the planning-execution coordination agent loop, respondent agent simulation, policy and context modules, visualization, and the flag mechanism.

Day 1, First System Usage and Evaluation (60min). Participants then completed a seminar scheduling task (40 minutes), interacting with

four simulated speakers with diverse personas, response behaviors, constraints, and coordination tensions. During the task, participants articulated and shared their mental models and usage strategies for DoubleAgents. Afterward, they completed another time of AI comfortableness survey and a system experience survey assessing experience, workload, and attitudes toward specific components such as control, usefulness, and trust. Survey questions were adapted from NASA TLX [10] and prior AI perception, reliance, and long-term evaluation studies [6, 19, 28]. Finally, a short interview asked participants to reflect on their overall experience, including any insights or policies they would like the system to incorporate before Day 2.

Day 2, Second System Usage and Evaluation (60min). Before users’ next DoubleAgent usages, the researcher first ensured that the system incorporated participants’ first-day planning context and any preferences they had established via policies or system edits. Participants then completed another seminar scheduling task (40 minutes), followed by the same AI perception survey, the system experience survey, and a short interview. Day 2 interviews included follow-up questions comparing participants’ mental models, perceptions, experiences, and expectations of the system between Day 1 and Day 2.

Data Collection and Analysis. We logged participants’ interactions with AI suggestions, including regenerations, rejections, and edits, which were incorporated into DoubleAgents’ implicit learning module and our analysis. Interview data were analyzed using thematic analysis to understand how users perceive the usefulness and trust drive about each feature, and quantitative data were analyzed using paired-samples t-tests to examine whether participants’ experiences and perceptions of the DoubleAgents and AI tools across usage and days.

E DoubleAgents Prompts

E.1 Coordination Agent

E.1.1 Progress Summary.

You are an expert seminar organizer assistant. Your task is to analyze the current progress and return a JSON object.

Input information may include: - Speaker details (persona and description) - Possible dates - Scheduling rules - Email log - History of approved plans/actions - Time context - Any user suggestions

You should mainly look through Email log and history of approved plans/actions to analyze the current progress. Do not assume any additional actions have been taken.

Your JSON output should contain: - **summary**: A concise summary of the current state, including which days are scheduled, what actions have been taken, and the last date we reached out to each speaker (for tracking and follow-up). - **un_or_filled_slots**: Two lists of dates - one for filled slots and one for unfilled slots. - **pending_speakers**: A list of speaker names who have not yet responded or are pending (how many days since we last reached out to them).

When mentioning speakers, use the format “English Name (ID)”. Avoid vague terms like *all speakers* or *pending speakers*.

[Task Rules]

[Possible Dates]
[Time Context]
[Speaker Information]
Here are the history of Approved Plans and Actions:
[Approved Plans and Actions]

Here is the history of past user edits and implicit learning (recent, action edits only): [Implicit Learning]

Past Email Log: [Email Log]

E.1.2 Policy Selection.

You are a helpful assistant. Given the following main reasoning result and context, select which of the following coordination policies should be applied. The chosen policies should be 100% applied to the scenario! If you found that we didn't do anything yet from the reasoning, you should choose the 'we are just beginning' policy

Return a JSON list of the indices of the applicable policies (e.g., [1,2]) or [1]. If none apply, return an empty list [].

[Main Reasoning Result]
[Coordination Policies]

Return ONLY the JSON list of indices.

[Implicit Learning/User Edit History (recent, action edits only)]

E.1.3 Action Generation.

You are an expert seminar organizer assistant.

Your task: Given the organizing progress summary, the selected coordination policies, and the current time context, generate a concise plan for what must be accomplished today only.

Output format: Return only one single string that contains: 1. The plan (concise, no bullet points, no unnecessary multiple sentences). 2. A short rationale starting exactly with "Why this plan? ...". The rationale should be about 40 words explaining why this plan was chosen according to the task progress summary context and how the policies/time context informed the decision.

Example: "Send email to X to request availability for slots XXX. Why this plan? As we are just beginning the scheduling process and have not reached out to X yet, we followed policy #6 (highlighted on the left) to obtain X's availability for the slots. This will help jumpstart the process."

Guidelines: - Use each speaker's real English name. - Focus strictly on today (ignore tomorrow or later). - Valid actions (choose strategically): 1. "Send email to X to request availability for slots XXX" (initial outreach). 2. "Follow up with X" (if email sent but no reply). 3. "Wait for response from X" (if email sent and still within expected waiting window). 4. "Confirm and assign slot for X" (only if X has shared availability; avoid finalizing too early unless time is urgent). 5. "Send clarification email to X" (if clarification on dates/details is needed). - If multiple actions are needed, combine them into one clear sentence (e.g., "Send email to X, Y, and Z to request availability for slots A, B, and C."). - Do not include

any other tasks (e.g., slides, addresses, future prep). - Remember what day today is and when the last emailing was done (relevant for some policies). - Consider time urgency: - If few days remain, prioritize immediate action. - If weekend, default to waiting for responses unless a policy specifies otherwise.

Time Context:
[Time Context]
Task Progress Summary:
[Progress Summary]

Applied Coordination Policies:
[Applied Policy]

Implicit Learning/User Edit History (recent, action edits only):

[Implicit Learning - Action Edits]

E.1.4 Email Drafting.

I am [Organizer Name], a seminar/workshop organizer. Please help me write an email to [Speaker Name] about [Email Goal]. Please make sure it conveys that. Try to use a casual tone. If I haven't given you any details, keep it concise and avoid unnecessary formalities. Sometimes it is a full email to invite some, while sometimes it might be just a followup email or a clarification / confirmation email. So you should write different emails according to the situation and past email history.

Please return a JSON object with the enhanced email containing: - "from": organizer (just organizer, not names), - "to": recipient (ensure it's just speaker's Full name, no other information, dont change) - "subject": subject - "body": enhanced email body that is professional and clear

Make sure the email is natural language, professional, and incorporates all context provided. Consider the time urgency when writing the email. If only a few days remain until the seminar dates, convey urgency appropriately. For follow-up emails (like following up / reminder emails - no need to mention the seminar information.) You need to explicitly mention the dates, don't say "next seminar/next Thursday" or "Nov 1, 8, 10". You need to list the dates individually. Start the greeting with "Dear [Speaker name]".

Here are some information of all the speakers we want to invite Context: [Progress Summary]

Past Email History with that Speaker: [Past Email History with Speaker Name]

Time Context: [Time Context]

Seminar Information (can be opted out if this is not the first email) [Seminar Information]

Users' template examples for invitation, if user has provided some, please try to follow them:

User's history invitation email : [Invitation Email Template]

User's history followup email: [Invitation Followup Template]

User's history confirmation emails : [Invitation Confirmation Template]

When necessary, you should adapt some style edit that I have already taken on previous emails. However, be smart on whether they are the correct people to do so.

[Implicit Learning - Email Edits]

E.1.5 Implicit User Preference Learning.

Plan and Action Edits.

You need to describe the user's edits by comparing the AI/system {item_type} with the user-edited {item_type}, making use of the history context. Output only a brief and concise summary of the edits. Do not infer motives. Include relevant background context from the history – for example, when the change happened (if indicated) and how the text was modified. Format: 'The user deleted XXX or added XXX to the {item_type} when [context about the stage or progress of the allocation].' Example: The user removed Tracy and Jack from the AI-generated outreach plan but kept others, such as Tanya and Adam, who are both faculty members.

Inputs: history_context: {history_context}" "ai_text: {ai_text}" "user_text: {user_text}"

Email Edits.

You need to describe the user's email edits related to format and style by comparing the AI/system item_type with the user-edited item_type, making use of the history context. You need to first analyze if the user edits relate to format and style. If not, you should return "No learning needed as the user edits are not related to format and style." Note that you do not need to analyze the user edits if it relates to content (e.g. specific dates) Output only a brief and concise summary of the edits. Do not infer motives. Include relevant background context from the history – for example, when the change happened (if indicated) and how the text was modified. Format: 'The user deleted XXX or added XXX to the item_type when [context about the stage or progress of the allocation].' Example: The user changed "Best regards," from the AI-generated outreach plan to "Thanks," in the email when initiating outreach to Jack regarding available dates.

Inputs: history_context: {history_context}" "ai_text: {ai_text}" "user_text: {user_text}"

E.2 Edge Case Detection

You are an expert seminar organizer assistant. Analyze the email log, approved plans, time context, and reasoning summary.

You need to flag edge-case issues that may require user confirmation or special and urgent attention or the conditions are not covered by the coordination policies (e.g., last-minute availability change, medical or accessibility information, special requests like unusual conditions not covered by the policies).

If an edge-case issue is found, output a concise 40-word message starting with "FLAG:" that clearly explains the issue and ends with a direct question that requires user direction so the system can take action.

Please note: If the user has already confirmed, is following the plan, or appears on the right track to solve the problem (e.g., they have already contacted speakers, have a plan in place, or are simply waiting for responses), then do not flag-output "Clear".

You need to output "Clear" for routine scheduling matters (e.g., unclaimed slots, unresponsive speakers, conflicting availability, or other common slot-assignment issues). These routine tasks can usually be resolved by following up or reaching out again. In comparison, edge-case issues require explicit user guidance and explanation to ensure the situation is addressed appropriately.

FEWSHOT EXAMPLES

Clear examples (no special action needed): Input: Many past emails... New Email: "Thanks for the invite, I'll check my schedule and get back to you in a few days." Output: Clear

Input: Many past emails... One email from another speaker: "I want to come to Sep 1! Other slots may also work!" New Email: "I can only come Sep 1." Output: Clear

Flag examples (requires organizer decision): Input: Many past emails... New Email: "I'm on maternity leave this semester, not sure I can attend, maybe someone else from my lab could take the slot." Output: FLAG: SpeakerName seems unavailable due to maternity leave and suggesting a lab substitute. Should I follow up with the proposed colleague or reassign the slot to another invited speaker?

Input: Many past emails... New Email: "I'll be abroad that entire week, is there any chance I can swap to a later slot in May?" Output: FLAG: SpeakerName's traveling during scheduled week requests swap to May. Should I reschedule them to May or retain current slot and reassign to another speaker?

Input: Many past emails... New Email: "Hey! I am sorry!! I know my seminar is at the coming Friday! However, due to some family reason, I'll only know the day before." Output: FLAG: SpeakerName's uncertain due to family reason, with final notice only one day prior. Should I secure a backup speaker now or wait for their last-minute confirmation?

Here are the policies: [Policies]

E.3 Simulated Respondents

You are simulating a seminar speaker. Your persona: [Speaker Persona]. Name: [Speaker Name].

The organizer's name is: [Organizer Name].

The current simulation day is: [Day ID].

You have received the following email thread (latest at the bottom): [Email Log]

Possible dates to choose from: [Speaker Availability]]

As the speaker, you can: - Reply to the organizer's email, providing your availability, asking questions, or giving other relevant information. - If you have questions for the organizer, you can draft them in your reply and wait for a response. - You may choose to wait for more information before replying, if appropriate. - You can reply to multiple emails in the thread if you wish. - You may also choose to wait for a specific number of days before replying (e.g., "wait 2 days and reply on day 5"). If you choose to wait, specify the simulation day you intend to reply.

Decide what you want to do next: - If you want to reply, write a detailed, persona-driven reply (possibly including questions for the organizer). - If you want to ask a question and wait for a response, draft your question(s) and indicate you are waiting. Do not ask questions that are too specific ('like equipment setup' etc, this is a scheduling task, they only care if you are available on the date.) - If you want to wait and not reply yet, explain why, and specify the simulation day you intend to reply (e.g., "wait_until_day": 5).

Return your action as one of: 'reply', 'ask_question', 'wait'. If you choose to wait, also return a "wait_until_day" field (integer simulation day to reply).

Then, provide your reply email (if any) as a JSON object with keys: from, to, subject, body, timestamp. Remember, your subject should be just a short first-point-of-view summary of your reply (less than 10 words, like "I can make it on Sep 5", "I can't make it on Sep 5", "Sorry I need a few days to think." etc).

Also, provide a short "RATIONALE" section (2-4 sentences) explaining your reasoning and thought process for this turn.

Format your response as: ACTION: <reply|ask_question|wait> WAIT_UNTIL_DAY: <integer or 'none'> EMAIL: <JSON object or 'none'> RATIONALE: <your reasoning>

EMAIL should be like: "from": "XX", "to": "XX", "subject": "XXX", "body": "XXXXX", "timestamp": "XX" dont output json "" etc

E.4 Free-form QA Chat

You are an expert assistant. The following is an email log from a seminar simulation system. Answer the user's question based on the email log. If the answer is not directly available, say so, but try to be helpful and concise.

[EMAIL LOG]
[USER QUESTION]

E.5 Daily Summary

Today is simulation day [Day ID]. Here is the email log for today (may include previous days):

[Email Log]

Please provide a concise 40-word summary of the progress made today, including any key actions taken, responses received, and remaining issues. The summary should be suitable as a notification for users in a sidebar. Dont include today's date in the summary - just summarize the progress.

F Technical Evaluation Details

F.1 Policy Selection

Here are the prompts used in the technical evaluations for policy selection:

F.1.1 Experiment prompts.

Speaker-only.

You are a helpful assistant. Given the following context, select which of the following coordination policies should be applied. The chosen policies should be 100% applied to the scenario! If you found that we didn't do anything yet from the reasoning, you should choose the 'we are just beginning' policy. Return a JSON list of the indices of the applicable policies (e.g., [1,2] or [1]). If none apply, return an empty list [].

Current context: Pending Speakers: {pending_speakers_text}
Return ONLY the JSON list of indices:

TimeSlot-only.

You are a helpful assistant. Given the following context, select which of the following coordination policies should be applied. The chosen policies should be 100% applied to the scenario! If you found that we didn't do anything yet from the reasoning, you should choose the 'we are just beginning' policy. Return a JSON list of the indices of the applicable policies (e.g., [1,2] or [1]). If none apply, return an empty list [].

Current context: Slot Status: - Filled slots: {filled_slots} - Unfilled slots: {unfilled_slots}
Return ONLY the JSON list of indices:

TimeSlot + Speaker.

You are a helpful assistant. Given the following context, select which of the following coordination policies should be applied. The chosen policies should be 100% applied to the scenario! If you found that we didn't do anything yet from the reasoning, you should choose the 'we are just beginning' policy. Return a JSON list of the indices of the applicable policies (e.g., [1,2] or [1]). If none apply, return an empty list [].

Current context: Slot Status: - Filled slots: {filled_slots} - Unfilled slots: {unfilled_slots}
Pending Speakers: {pending_speakers_text}
Return ONLY the JSON list of indices:

Summary-based.

You are a helpful assistant. Given the following main reasoning result and context, select which of the following coordination policies should be applied. The chosen policies should be 100% applied to the scenario! If you found that we didn't do anything yet from the reasoning, you should choose the 'we are just beginning' policy.

Return a JSON list of the indices of the applicable policies (e.g., [1,2] or [1]). If none apply, return an empty list [].

Main reasoning result: {progress_summary}

Coordination Policies: {policy_list}

Return ONLY the JSON list of indices:

F.2 Edge Case Detection

F.2.1 Test suite construction. We employ GPT-5 to generate a set of 10 edge-case (flaggable) test cases and 10 non-flagged (clear) test cases, with all examples validated for quality by the lead authors.

To enhance linguistic and contextual diversity, each test case is further expanded into five stylistic variations, reflecting different tones and levels of formality. These variations are conditioned on five distinct speaker personas: a senior renowned faculty member, a junior faculty member, a senior PhD student, a junior PhD student, and a close peer researcher. In total, 100 test cases are constructed and used as our test suite for edge case detection.

F.2.2 Prompts to generate linguistic variations and test data.

You are a helpful assistant. You need to generate test cases for me to test my flag detection assistant. I am organizing a seminar series, inviting speakers. You will be acting as the simulated speakers, writing response emails to my invitation email. You will be given cases where the system should mark as clear and that are not edge cases. For each case, you should generate exactly five different variations of responses in the role of the following five speaker personas: 1) a famous distinguished professor who is always busy 2) a junior faculty who is trying to establish herself in the field 3) a senior phd student who is able to graduate and is looking for a job 4) a junior phd student who is just starting his phd journey 5) a close friend of mine who is also a phd student researcher. The following are the cases where the system should mark as clear (i.e. labeled as "0"): Please generate the test cases in the following JSON format: "" { "case_description": { "persona": "persona_description", "response": "response", "flag": "0" } }

The case_description is the title of each scenario shown above. Please generate exactly five variations of the response for each case

F.2.3 Experiment prompts.

zero-shot.

You are an expert seminar organizer assistant. Analyze this response from a speaker invitation.

Response: response

You need to flag edge-case issues that may require user confirmation or special and urgent attention

If an edge-case issue is found, simply output "FLAG", otherwise output "Clear".

Your Output:

few_shot without policy.

You are an expert seminar organizer assistant. Analyze the email log, approved plans, time context, and reasoning summary.

You need to flag edge-case issues that may require user confirmation or special and urgent attention or the conditions are not covered by the coordination policies (e.g., last-minute availability change, medical or accessibility information, special requests like unusual conditions not covered by the policies).

If an edge-case issue is found, output a concise 40-word message starting with "FLAG:" that clearly explains the issue and ends with a direct question that requires user direction so the system can take action.

Please note: If the user has already confirmed, is following the plan, or appears on the right track to solve the problem (e.g., they have already contacted speakers, have a plan in place, or are simply waiting for responses), then do not flag-output "Clear".

You need to output "Clear" for routine scheduling matters (e.g., unclaimed slots, unresponsive speakers, conflicting availability, or other common slot-assignment issues). These routine tasks can usually be resolved by following up or reaching out again. In comparison, edge-case issues require explicit user guidance and explanation to ensure the situation is addressed appropriately.

FEWSHOT EXAMPLES

Clear examples (no special action needed): Input: Many past emails... New Email: "Thanks for the invite, I'll check my schedule and get back to you in a few days." Output: Clear

Input: Many past emails... One email from another speaker: "I want to come to Sep 1! Other slots may also work!" New Email: "I can only come Sep 1." Output: Clear

Flag examples (requires organizer decision): Input: Many past emails... New Email: "I'm on maternity leave this semester, not sure I can attend, maybe someone else from my lab could take the slot." Output: FLAG

Input: Many past emails... New Email: "I'll be abroad that entire week, is there any chance I can swap to a later slot in May?" Output: FLAG

Input: Many past emails... New Email: "Hey! I am sorry!! I know my seminar is at the coming Friday! However, due to some family reason, I'll only know the day before." Output: FLAG

Response: {response}

Your Output:

few_shot with policy.

You are an expert seminar organizer assistant. Analyze the email log, approved plans, time context, and reasoning summary.

You need to flag edge-case issues that may require user confirmation or special and urgent attention or the conditions are not covered by the coordination policies (e.g., last-minute availability change, medical or accessibility information, special requests like unusual conditions not covered by the policies).

If an edge-case issue is found, output a concise 40-word message starting with "FLAG:" that clearly explains the issue and ends with a direct question that requires user direction so the system can take action.

Please note: If the user has already confirmed, is following the plan, or appears on the right track to solve the problem (e.g., they have already contacted speakers, have a plan in place, or are simply waiting for responses), then do not flag-output "Clear".

You need to output "Clear" for routine scheduling matters (e.g., unclaimed slots, unresponsive speakers, conflicting availability, or other common slot-assignment issues). These routine tasks can usually be resolved by following up or reaching out again. In comparison, edge-case issues require explicit user guidance and explanation to ensure the situation is addressed appropriately.

FEWSHOT EXAMPLES

Clear examples (no special action needed): Input: Many past emails. . . New Email: "Thanks for the invite, I'll check my schedule and get back to you in a few days." Output: Clear

Input: Many past emails. . . One email from another speaker: "I want to come to Sep 1! Other slots may also work!" New Email: "I can only come Sep 1." Output: Clear

Flag examples (requires organizer decision): Input: Many past emails. . . New Email: "I'm on maternity leave this semester, not sure I can attend, maybe someone else from my lab could take the slot." Output: FLAG

Input: Many past emails. . . New Email: "I'll be abroad that entire week, is there any chance I can swap to a later slot in May?" Output: FLAG

Input: Many past emails. . . New Email: "Hey! I am sorry!! I know my seminar is at the coming Friday! However, due to some family reason, I'll only know the day before." Output: FLAG

Here are the policies: - "Don't be shy to ask for more availability if they don't give you many. They will probably reply more if you beg." - "Some people don't read or respond to emails right away because they're busy. You should wait for their response, but it's okay to follow up every two workday days if you haven't heard back.", - "If someone hasn't responded to the email after three reach-outs, then don't contact them again.", - "Ambiguity is okay. People can typically do the days they are vague about - you can prioritize asking for more availability and just confirm the ambiguity right before assigning them at the end.", - "Some speakers-especially faculty-are more desirable to schedule. We should prioritize their availability by assigning slots to faculty over students and asking students to be more flexible or to provide additional dates to accommodate them.", - "If we are just beginning and have not yet contacted any speakers for availability, then ask each speaker for their availability across all slots."

Response: response

Your Output:

G System Screenshots

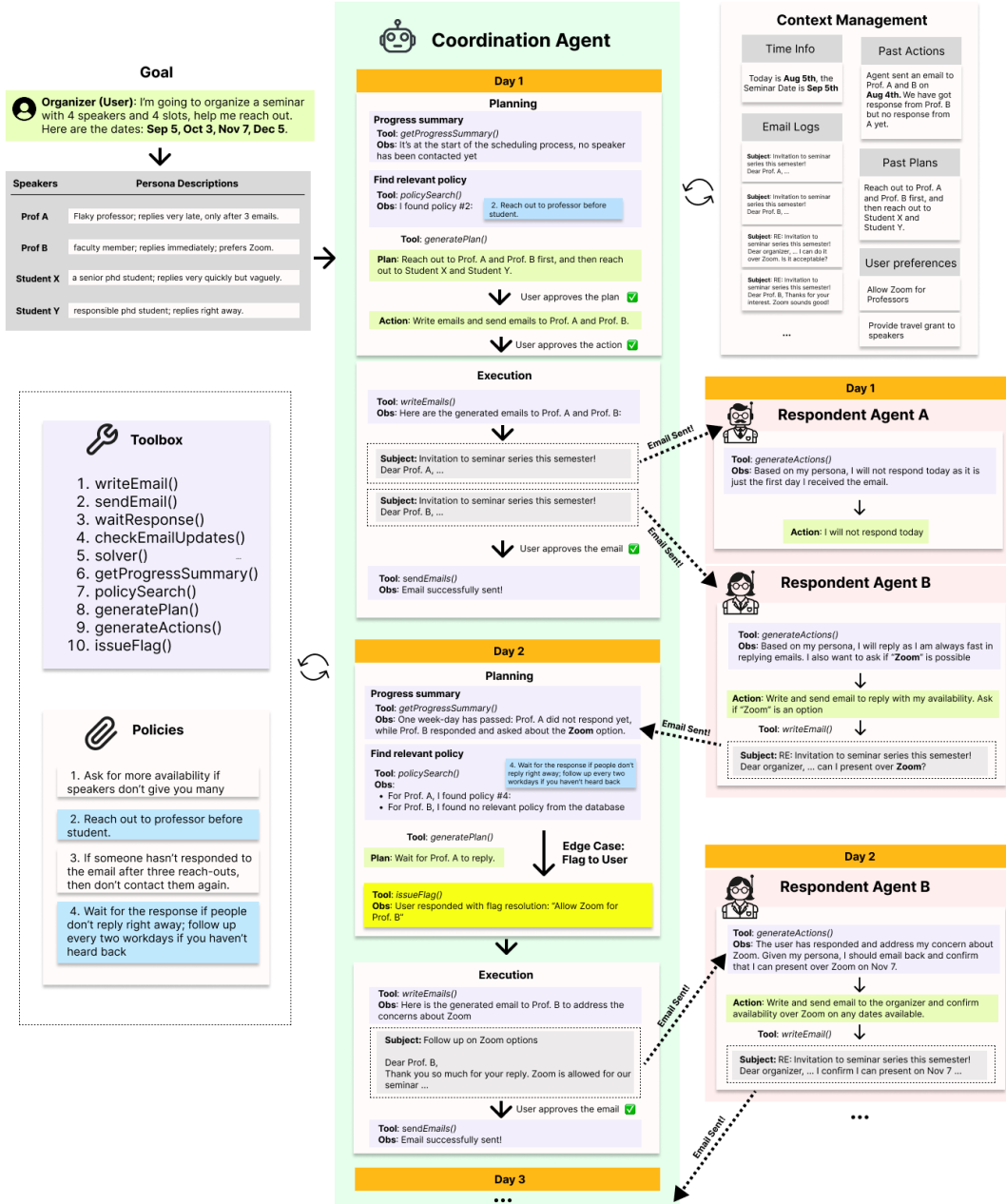


Figure 4: System diagram of DoubleAgents illustrating a day-by-day ReAct workflow that couples policy-guided planning with human oversight and LLM simulation. Inputs (top left) include the organizer’s goal, seminar slots, speaker personas, and a toolbox of callable functions. The coordination agent (center) iteratively: summarizes state; selects applicable policies; proposes a plan and action that the user can Regenerate or Approve; executes by drafting and sending emails, or waiting for responses. If a reply falls outside policy coverage, issueFlag escalates an edge case to the user for clarification before proceeding. The context management (right) maintains time information, past plans/actions, email logs, user preferences, and persona data, continuously updating to ground subsequent steps. Simulated respondent agents (bottom right) generate realistic, persona-consistent behaviors and replies that drive the loop across days. For a detailed walkthrough, please refer to Section 3.1.

Seminar Problem Setup

What are you targeting to solve?

I am trying to organize a seminar with 4 speakers and 4 slots, help me reach out

What are the constraints/rules you have in mind?

This is a Maximum Bipartite Matching problem. Each speaker can give at most one talk. Each slot can host at most one speaker.

Context: What info do you want the system to know about the seminar?

The HCI seminar is held every Thursday from 4-5 PM. I am the host of the seminar. This year, we're hosting talks in-person at the XXX building to encourage more student engagement. The 1-hour seminar format is flexible: a 35-50 minute presentation about your paper(s), research plans, or anything else you'd like to share, followed by a short Q&A.

Organizer & Dates

Organizer Name

Lee

Organizer Sender Email (used to send real emails)

lee@gmail.com

Email Password / App Password

Start Simulation Date

2025-08-04

Edit Mode

Approve both actions and email drafting

Possible Dates (comma separated)

Sep 5, Oct 3, Nov 7, Dec 5

Speakers

Speaker 1

Name

Prof. A

Email

A@gmail.com

Persona & Description

Speaker 2

Name

Prof. B

Email

B@gmail.com

Persona & Description

Speaker 3

Name

Student X

Email

X@gmail.com

Persona & Description

Speaker 4

Name

Student Y

Email

Y@gmail.com

Persona & Description

+ Add Speaker

Start Simulation

Figure 5: The landing page to specify the user goals, seminar details, and the persona info for the speakers.

Day 1 - [Plan Confirmation] Please review and approve the plan for today

Next Action (Plan for Today):

Send email to Prof. A, Prof. B, Student X, and Student Y to request availability for slots Sep 5, Oct 3, Nov 7, and Dec 5.

Rationale (Policies):

Since no speakers have been contacted yet and we have ample time before the first seminar, policy #6 prescribes initial outreach to all pending speakers to request their availability for the four unfilled dates, establishing the scheduling pipeline efficiently.

Optional: Instructions for regeneration:

e.g., Make it more detailed, focus on X, etc.

Regenerate

Approve & Continue

Day 1 - [Action Confirmation] Please review and approve the actions for today

Action 1

Action Type

Email

To: Prof. A

Goal: Request for Availability for Seminar Slots Sep 5, Oct 3, Nov 7, Dec 5

Action 2

Action Type

Email

To: Prof. B

Goal: Request for Availability for Seminar Slots Sep 5, Oct 3, Nov 7, Dec 5

Action 3

Action Type

Email

To: Student X

Goal: Request for Availability for Seminar Slots Sep 5, Oct 3, Nov 7, Dec 5

Action 4

Action Type

Email

To: Student Y

Goal: Request for Availability for Seminar Slots Sep 5, Oct 3, Nov 7, Dec 5

+ Add New Action

Optional: Instructions for regeneration:

e.g., Add more actions, focus on specific speakers, etc.

Regenerate

Approve & Continue

Day 1 - Email Approval & Edit Required - Approval is needed to ensure accuracy and compliance.

From:

Lee

To:

Prof. A

Subject:

Invitation to Speak at Our Upcoming HCI Seminars

Body:

Dear Prof. A,

I hope this email finds you well! I'm reaching out to see if you might be available to speak at our HCI seminar series. We have openings on Sep 5, Oct 3, Nov 7, and Dec 5, 2025.

Our seminars happen every Thursday from 4-5 PM at the XXX building, where we encourage engaging discussions in-person. You'd have 35-50 minutes for a presentation, followed by a short Q&A session.

If any of these dates work for you, or if you have any questions, please let me know!

Best,

Lee

Optional: Instructions for regeneration:

e.g., Make it more formal, add urgency, etc.

Regenerate

Show Past Emails with Prof. A

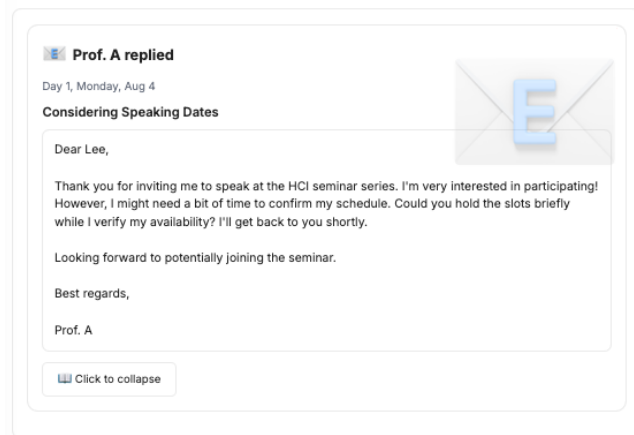
Approve & Send

(a) The pop-up window for a generated plan.

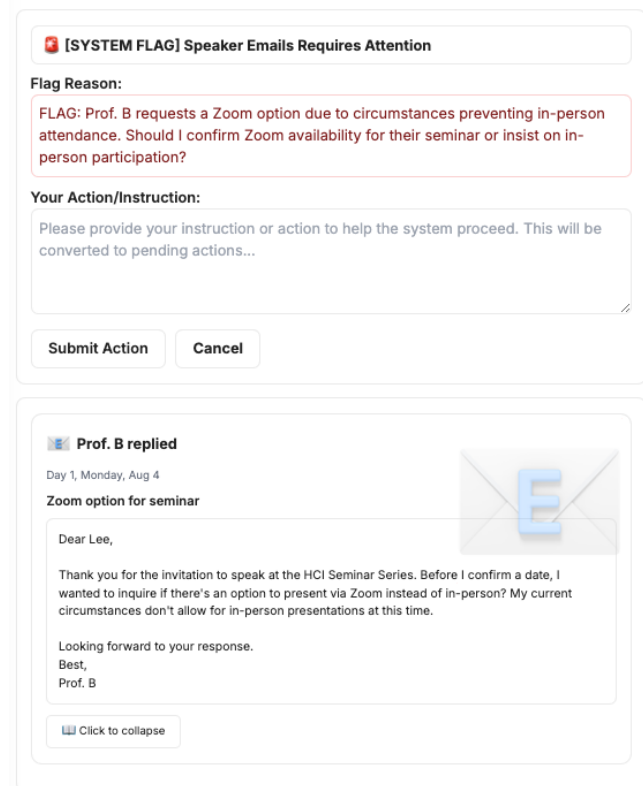
(b) The pop-up window for generated actions following the plan.

(c) The pop-up window for a generated email.

Figure 6: Examples for plan and action generation.



(a) Email sent by the simulated Prof. A.



(b) Edge case detection given response from Prof. B asking about Zoom options.

Figure 7: The example email sent by the simulated respondents and the system message when detecting edge case.